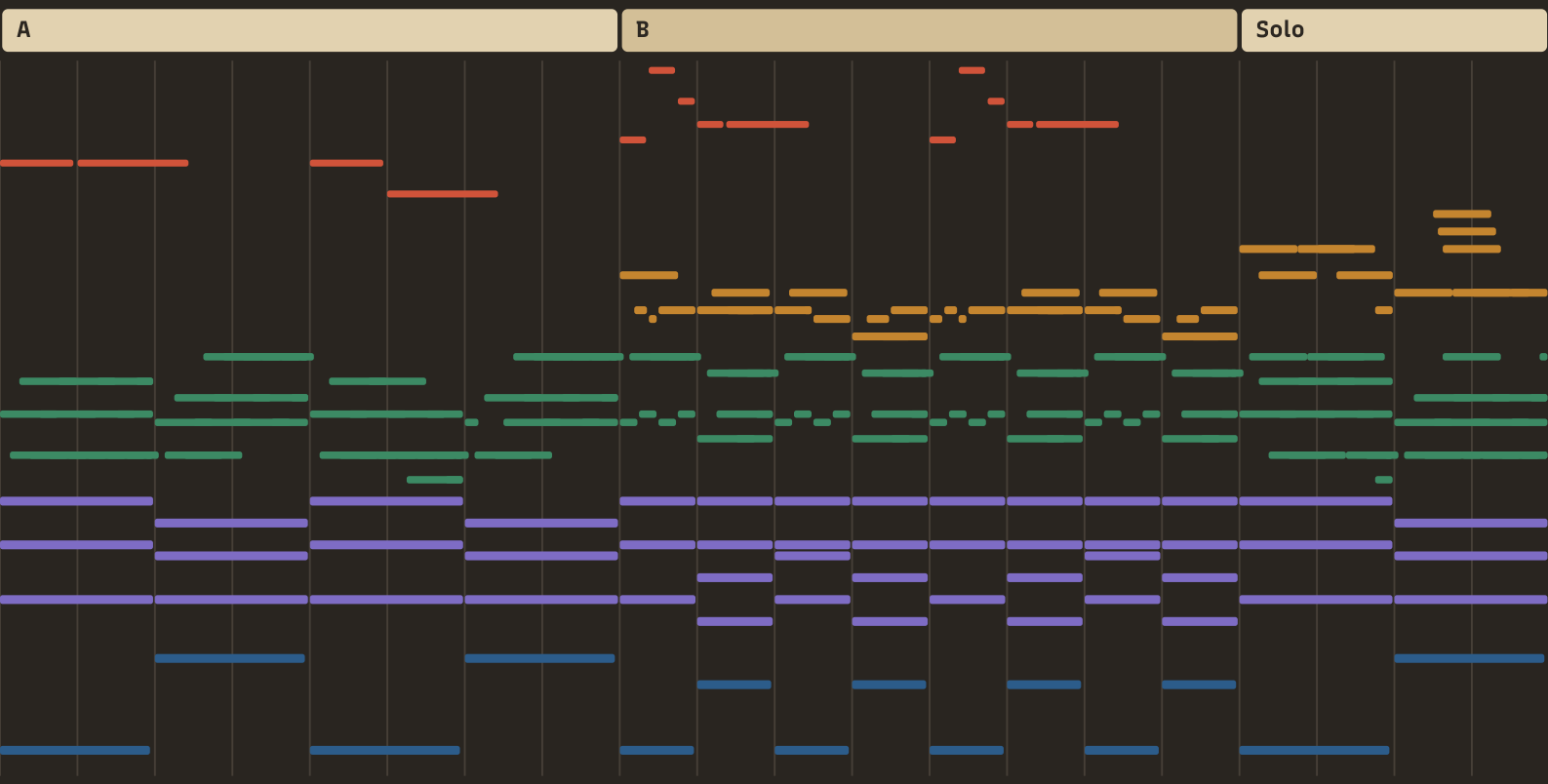
Squimble Music Maker

A procedural music generator for the web, built on music theory rather than AI, to score a video game



Ahsan Siddiqui

A technical report, BSc (Hons) Computer Science

Cover: the first twenty bars of "mossy-lantern" in the Nostalgic mood, drawn from the notes the composer writes for it

October 2026

Contents

1 Introduction 8
- 1.1 Background 8
- 1.2 Aims 9
- 1.3 Objectives 9
- 1.4 Report Organisation 10

2 Literature review 11
- 2.1 Algorithmic Composition 11
- 2.2 Procedural and Adaptive Music in Games 12
- 2.3 Neural Music Generation 13
- 2.4 Music Theory Foundations 13
- 2.5 Digital Sound Synthesis 14
- 2.6 Gaps in Research and Justification for This Project 15

3 Design 16
- 3.1 Overview of design goals 16
- 3.2 System structure 17
- 3.3 The song model 18
- 3.4 User interface design 19
- 3.5 Design iterations and improvements 23
- 3.6 Ethical and licensing considerations 25

4 Implementation 27
- 4.1 Random numbers and seeds 27
- 4.2 Harmony 29
- 4.3 Melody 31
- 4.4 Counterpoint 32
- 4.5 Rhythm, groove and feel 34
- 4.6 The band listens 36
- 4.7 Arrangement and orchestration 36
- 4.8 Sound synthesis 37
- 4.9 Building the sample library 46
- 4.10 The learn tab: a guitarist's hand and the Viterbi algorithm 47
- 4.11 Saving, sharing and keeping old songs 51

5 Testing and Evaluation 52
- 5.1 Overview of the testing approach 52
- 5.2 Automated checks 52
- 5.3 Performance 53
- 5.4 Feedback and evaluation against the objectives 54
- 5.5 Limitations 55

6 Conclusion 56

References 58

Appendices 60

A Moods and flavours 60

B Glossary 62

Acknowledgement

I would like to thank my friends who tried the music maker on their phones and told me honestly that it was complicated and cluttered. Their feedback led directly to the layout that the page has now, and this project would be much harder to use without them.

Additionally, I would like to thank the people behind the free sample libraries that the band plays: Versilian Studios (VSCO 2 Community Edition and the Versilian Community Sample Library), Karoryfer Samples, the University of Iowa Electronic Music Studios, the FluidR3 and Musyng Kite soundfonts, and the many Freesound uploaders whose recordings of rain, birdsong, singing bowls and an amp feeding back give the songs a sense of place. Without people giving their recordings away for free, none of the recorded instruments in this project would exist.

I would also like to thank the fans who transcribe songs on Songsterr, whose tabs made it possible to study about one hundred and fifty shoegaze songs as data.

Abstract

This report presents the design and development of Squimble Music Maker, a web based music generator that writes complete songs from a two word seed using the rules of music theory instead of machine learning. The project was built to provide the soundtrack for Squimble Quest, a game I am developing, and needed to produce music that is varied, editable, small enough to ship in a browser and identical every time the same seed is played. The system is split into a composer that turns settings into notes using plain mathematics, and a band that plays those notes through synthesisers built on the Web Audio API and recordings of real instruments. This report covers the research behind the project, the music theory and signal processing behind each part of it, the mathematics of its algorithms (from Karplus-Strong string synthesis to a damped spring model of a singer's pitch and a Viterbi search for guitar fingerings), and how the project has been tested and evaluated.

Figures

1 The Squimble Music Maker page, showing a song generated from the seed "mossy-lantern" in the Nostalgic mood 9

2 A comparison of the main approaches to algorithmic composition and how each is used in this project 12

3 A system diagram outlining the files of the music maker and how they depend on each other 17

4 The settings generated for the seed "mossy-lantern" in the Nostalgic mood and Theme style (shortened) 18

5 The order of the picks that write a song's settings, and the steps that turn those settings into notes 19

6 The score, showing each layer's notes across the song's sections, with chord names above 20

7 The mood, flavour, style, modifier and remix controls 20

8 The Song tab, with chords written as roman numerals for parts A, B and C, and the chords they make in this key below 21

9 The Band tab, showing the melody layer's sound, pattern, mix and note editor 21

10 The Arrange tab: a grid of which part each layer plays in each section, with the picked section's settings and note editor below 22

11 The music maker on a phone sized screen (390 pixels wide) 23

12 A selection of rows from the project's "what sounded wrong, and the fix" table 25

13 hashText() and makeRng(): the seeded random numbers behind every choice the composer makes 28

14 CHORD_MOVES as probabilities: how likely each chord of the scale is to lead to each other chord in a Markov progression 30

15 voiceChord(): choosing the inversion and octave of a chord that moves least from the one before 31

16 The fourteen melodic contours in CONTOURS, each drawn for eight notes (scale steps above or below the first note) 31

17 CONSONANCE: the cost of each interval between the counter melody and the tune above it. Thirds and sixths are free, while seconds, sevenths and the tritone are expensive 33

18 The heart of counterpoint(): Fux's rules of counterpoint written as a cost for every candidate note 34

19 Part of GROOVES: drum charts written as text, read by the drums, the bass and the chord stabs together 35

20 How feel() moves a note within its beat for different amounts of swing 35

21 The NES style 25% pulse wave (WAVES.pulse25): the strength of its first twenty harmonics, with every fourth missing, and the wave they add up to 38

22 A plucked A2 (110 Hz) from ksBuffer(): its first 30 milliseconds, in which the same burst of softened noise comes round every 9.1 ms (one cycle of the note), and its level over two seconds against the straight line of an exact 60 dB fade in 2.5 seconds 39

23 ksBuffer(): Karplus-Strong string synthesis, with the tuning error measured and a finger's triangle mixed into the pluck 40

24 The spectrum of the first 46 ms of a marimba's A3 from modalBuffer(), with modes at 220 Hz, 880 Hz (four times the note) and about 2180 Hz (9.9 times, the quickest to fade), above the soft noise of the mallet 41

25 Part of modalBuffer(): each mode is a two pole resonator, with stretched overtones and a detuned second string 42

26 The pitch paths produced by singerMove() for a step and an octave: a small preparation dip, a damped spring onto the note, and an overshoot that settles 43

27 singerMove(): a damped spring's step response used as the path of a singer's pitch, in cents 44

28 The mixing desk: each layer's channel, its sends to the shared room, reverb and echo, and the master chain 45

29 The tape saturation curve, tapeCurve(): unchanged near zero, rounding off the peaks, and slightly lopsided for warmth 46

30 pitchAt() in tools/build-samples.js: the YIN pitch detector used to check every recorded note 47

31 The Learn tab showing a guitar part: the fingers for the current chord on the neck, the scrolling tab, and the chord shapes used in the part 48

32 An illustration of the Viterbi search in the learn tab: every way to hold each chord, with the cheapest path through the part chosen at once 49

33 The Viterbi search in frettings() (learn.js), choosing strings and frets for a whole part at once 49

34 The Learn tab's sheet music view, with the key signature chosen from the song's scale and notes tied across bar lines 50

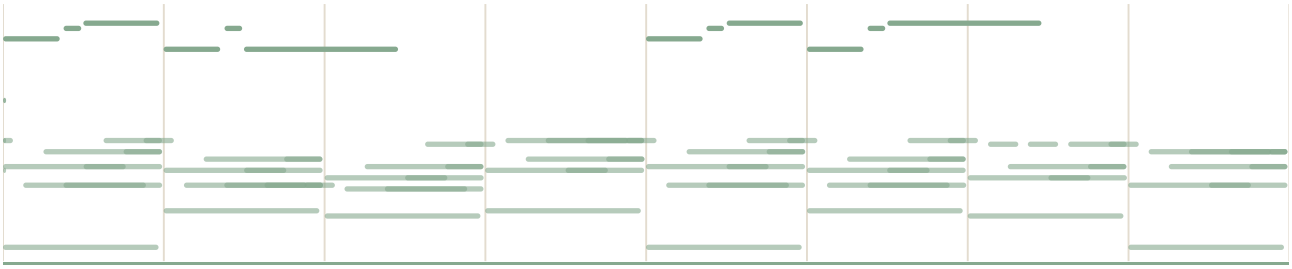
35 The Learn tab's piano view, with notes falling onto the keys: green for the right hand and blue for the left 50

36 The project's automated checks and measurement pages 53

37 The output of a few of the music check's blocks and the library check, run while writing this report 53

38 The effect of the performance changes on a large song, measured with Chrome's tracing of the real time audio thread 54

39 An evaluation of the project against the objectives set in section 1.3 55



Bars 33 to 40 of "mossy-lantern" in the Nostalgic mood, as the composer writes them

1 Introduction

1.1 Background

Music is one of the most important parts of a video game's atmosphere. A quiet piano over an empty field, a waltz in a busy town or a pounding drum line in a boss fight tell the player how to feel before anything else on the screen does (Collins, 2008). For a small, independent game made by a single developer, however, music presents a significant problem. Commissioning a composer is expensive, licensing existing music is restrictive, and writing dozens of tracks by hand takes a considerable amount of time and musical training. Likewise, a game with a large world needs a large amount of music, and short loops that repeat too often quickly become tiring for the player.

One answer to this problem is procedural, or algorithmic, music: music that is generated by a set of rules rather than written note for note. The idea is much older than computers. In the eighteenth century, musical dice games (the most famous being the *Musikalisches Würfelspiel* attributed to Mozart) let players assemble a minuet by rolling dice to choose between pre-written bars (Nierhaus, 2009). In 1957, Hiller and Isaacson used the ILLIAC computer at the University of Illinois to compose the *Illiac Suite*, a string quartet whose notes were chosen randomly and then filtered through the rules of counterpoint (Hiller and Isaacson, 1959). More recently, games such as *Spore* (2008) and *No Man's Sky* (2016) have used generative systems to produce music that responds to the player (Plut and Pasquier, 2020).

In the last few years, a second approach has become popular: generating music with neural networks trained on large amounts of recorded music, such as OpenAI's Jukebox (Dhariwal et al., 2020) and Google's MusicLM (Agostinelli et al., 2023). These models can produce impressive results, but they are expensive to run, they produce audio rather than notes that can be edited, they cannot be relied on to give the same output twice, and their training data raises serious questions of ownership. For a game that needs to run in a web browser, on a phone, with music that a player can change and save, these limitations are severe.

In response to these limitations, this project takes the older approach and builds on it. Squimble Music Maker writes music the way a composer following rules of thumb might: it picks a key, a scale and a tempo, writes chord progressions, grows melodies from short motifs, writes a second melody in counterpoint against the first, hands the tune around the band, and then has the band "listen" to each other so that nothing clashes. The music is then played by a band of synthesised and recorded instruments built entirely on the Web Audio API, in the browser, with no server. Figure 1 displays the page as it looks today.



Figure 1 The Squimble Music Maker page, showing a song generated from the seed "mossy-lantern" in the Nostalgic mood

1.2 Aims

The aim of this project is to create a web based music generator that can write complete, varied and pleasant songs from a seed using the rules of music theory, so that it can provide the soundtrack for the game Squimble Quest, while also letting people change, save, share and learn the songs it writes.

1.3 Objectives

The following are smaller objectives in the project that fall into the greater aim. They are specific criteria that will be completed as part of the main aim for the project.

- Generate a complete song (harmony, melody, accompaniment, drums and an arrangement) from a seed and a mood, the same way every time.
- Write the composer as plain mathematics with no page and no audio, so that it can be run in Node.js for testing and loaded into the game without changes.
- Build a band of instruments in the browser that sounds like real musicians playing in a room, rather than a General MIDI file.
- Let users change anything about a song, from the mood to individual notes, and save and share their songs without a server.
- Make sure that a saved song never changes, even as the composer is improved.
- Make the page usable on phones as well as computers, and teach people to play the parts that it writes.

1.4 Report Organisation

This report is broken down into six distinct chapters, all tackling a different aspect of the project.

1.4.1 Chapter 2 – Literature Review

This chapter reviews the history of algorithmic composition, procedural music in games, neural music generation, and the music theory and sound synthesis research that this project builds on. Gaps in current approaches are highlighted to justify the project.

1.4.2 Chapter 3 – Design

This chapter details the design of the system: its goals, its architecture and technology stack, the song model, the user interface, and how the design changed in response to songs that sounded wrong.

1.4.3 Chapter 4 – Implementation

This chapter explains how each part of the system works, technically and mathematically, with code taken directly from the project. It covers random numbers and seeds, harmony, melody, counterpoint, rhythm, sound synthesis, the sample library, the learn tab, and saving.

1.4.4 Chapter 5 – Testing and Evaluation

This chapter discusses how the project was tested, including its automated checks, the performance work done to stop songs stuttering, the feedback received, and the project's known limitations.

1.4.5 Chapter 6 – Conclusion

The concluding chapter of this report summarises and reflects on the overall project, acknowledges its limitations and proposes areas for future work.



Bars 5 to 15 of "riveted-koala" in the Medieval mood, as the composer writes them

2 Literature review

2.1 Algorithmic Composition

Algorithmic composition is the act of writing music using a formal set of steps, whether those steps are carried out by a person, dice or a computer (Nierhaus, 2009). Nierhaus groups the approaches into a few main families: stochastic methods, which choose notes by probability; Markov models, which choose the next event based on the events before it; generative grammars, which expand a musical structure from a set of rewriting rules; rule based or knowledge based systems, which encode what a composer knows about music; and machine learning, which learns its rules from examples.

The *Illiac Suite* is widely regarded as the first piece composed by a computer, and its method is surprisingly relevant to this project. Hiller and Isaacson generated notes at random and then rejected any note that broke the rules of first species counterpoint, such as parallel fifths or a melody that leapt too far (Hiller and Isaacson, 1959). This "generate and test" approach showed that a small number of rules could turn random numbers into something that sounds composed. However, it also showed the weakness of the approach: the music followed the rules but lacked any larger sense of structure, such as themes that return or sections that build.

Generative grammars attempt to address this weakness. Lerdahl and Jackendoff's *A Generative Theory of Tonal Music* (1983) describes how listeners hear music as a hierarchy, with notes grouped into motifs, motifs into phrases and phrases into sections. This suggests that a generator should work in the same way, building songs from the top down rather than choosing one note after another. Likewise, Huron (2006) argues that a large part of musical enjoyment comes from expectation: listeners enjoy predicting what comes next and being rewarded, or pleasantly surprised. Repetition is therefore not a weakness in music but a requirement, since a melody that never repeats gives the listener nothing to predict.

Markov chains are one of the most common tools in algorithmic composition. A first order Markov chain chooses the next chord based only on the current one, using a table of probabilities. Chord progressions in western

music follow strong habits (a dominant chord very often leads home to the tonic, for example), which makes Markov chains a natural fit for harmony (Piston and DeVoto, 1987). Their weakness is that they have no memory beyond the last step, so a long Markov melody wanders without direction.

Finally, Brian Eno's generative music offers a different way of thinking about the problem. For *Music for Airports* (1978), Eno recorded a handful of sung notes onto tape loops of different lengths and let them play together. Since the loops never lined up in the same way twice, the music kept changing while staying calm and coherent. Eno later described this as "generative music", music that is a system rather than a fixed recording (Eno, 1996). This idea has directly influenced the "drifting loops" bell pattern in this project, discussed in Chapter 4.

Approach	Strengths	Weaknesses	Used in this project
Stochastic (dice)	Simple, fast, endless variety	No structure or memory	Every choice, through seeded random numbers
Markov chains	Captures habits of harmony	Wanders over long spans	One of four ways chords are written
Grammars and hierarchy	Themes, phrases and sections	Hard to design by hand	Sections, motifs, question and answer phrases
Rule based	Explainable and editable	Rules must be found by ear	Counterpoint, voice leading, "listening"
Neural networks	Learns style from examples	Costly, opaque, not editable, licensing	Not used

Figure 2 A comparison of the main approaches to algorithmic composition and how each is used in this project

2.2 Procedural and Adaptive Music in Games

Games have experimented with music that changes with play for decades. LucasArts' iMUSE system, first used in *Monkey Island 2* (1991), let composers mark points in a piece where it could branch smoothly into another, so that the music followed the player from one area to the next (Collins, 2008). Collins describes two techniques that most adaptive game music still uses today: horizontal resequencing, where pre-written sections are reordered as the game changes, and vertical reorchestration, where layers of the same piece are added or removed to raise or lower its intensity.

Fully generative music is rarer. *Spore* (2008) used a procedural music system, with Brian Eno contributing to the design, to give each player's creatures and civilisations their own music. *No Man's Sky* (2016) built its soundtrack from stems recorded by the band 65daysofstatic, recombined by a system called Pulse as the player explored (Plut and Pasquier, 2020). In their review of generative music in games, Plut and Pasquier note that most systems either recombine pre-recorded material or generate music that is technically correct

but lacks the character of a composed score. They identify the need for systems that produce music of composed quality while still being controllable by designers.

Importantly, some of the most loved game soundtracks are not generative at all but have qualities that a generator could learn from. C418's music for *Minecraft* is sparse, with long silences between quiet pieces of piano and synthesiser, and the game only plays it now and then. This sparseness is part of why it does not become tiring, and it is one of the main references for the ambient moods in this project. Similarly, Toby Fox's music for *Undertale* relies heavily on short, memorable motifs that return in different forms across the whole game, and frequently stacks two melodies on top of each other.

2.3 Neural Music Generation

Neural models such as Jukebox (Dhariwal et al., 2020) and MusicLM (Agostinelli et al., 2023) generate audio directly from text descriptions or examples. Their results can be convincing, and they can imitate styles that would be very difficult to describe as rules. Despite this, they have several drawbacks for a project like this one. The first of many is cost: these models need powerful hardware to run, which rules out generating music live in a web browser on a phone. The second is control: they output finished audio rather than notes, so a user cannot change one chord or one note of a melody. The third is reproducibility: a game needs the same area to have the same music every time it is visited, and a song shared as a link needs to sound the same on another person's computer. Finally, there are ongoing legal and ethical questions about models trained on recorded music without the consent of the musicians who made it.

These drawbacks do not mean that neural models are a poor approach in general, but they suggest that for a small, editable, deterministic and offline system, a rule based approach is more appropriate.

2.4 Music Theory Foundations

Since this project writes music using rules, the rules themselves need to come from somewhere. Most of them come from traditional western music theory, which has spent centuries describing what composers do.

The first foundation is scales and modes. A scale is a selection of the twelve notes in an octave, and each mode (major, dorian, lydian and so on) gives a different feeling: lydian's raised fourth sounds bright and dreamlike, while phrygian's lowered second sounds dark and tense. Chords are built by stacking every other note of a scale, so a song's chords take on the colour of its mode (Piston and DeVoto, 1987).

The second foundation is functional harmony: the idea that chords have roles. The tonic (I) is home, the predominant chords (ii and IV) lead away from it, and the dominant (V) creates tension that wants to resolve back home.

Piston's table of "usual root progressions" describes how often each chord tends to follow each other chord, and it is effectively a Markov chain written in words.

The third foundation is voice leading: when chords change, each of their notes should move as little as possible. Tymoczko (2011) shows that this can be treated geometrically, with chords as points in a space and voice leading as the shortest path between them. This suggests a simple algorithm: try every way of arranging the next chord and keep the one with the smallest total movement.

The fourth foundation is counterpoint, the art of writing two or more independent melodies that sound good together. Fux's *Gradus ad Parnassum* (1725) set out rules that are still taught today: prefer consonant intervals such as thirds and sixths, move by step rather than by leap, prefer contrary motion (one voice rising while the other falls), and never move in parallel fifths or octaves, which makes two voices sound like one (Mann, 1965). These rules translate well into a cost function, which is how this project uses them.

2.5 Digital Sound Synthesis

Writing good notes is only half of the problem. The other half is playing them in a way that sounds like real instruments. There are several established techniques for this.

Additive synthesis builds a sound from sine waves at whole multiples of the note's frequency (its harmonics). Any repeating wave can be described this way (its Fourier series), and the strength of each harmonic largely decides the instrument's character. Frequency modulation (FM) synthesis, developed by Chowning (1973), creates rich, bell like sounds by modulating one oscillator's frequency with another, and was the basis of many digital synthesisers in the 1980s.

Physical modelling instead simulates how an instrument produces sound. The Karplus-Strong algorithm (Karplus and Strong, 1983) models a plucked string with a short burst of noise circulating through a delay line, softened slightly each time round. It is extremely cheap to compute and sounds surprisingly real, and Jaffe and Smith (1983) extended it with fine tuning, dynamics and string stiffness. Modal synthesis models struck objects (bars, bells, piano strings) as a set of resonant modes, each ringing at its own frequency and dying away at its own rate (Smith, 2010). Fletcher (1964) showed that the overtones of a stiff piano string are slightly sharper than whole multiples of the note, which is part of why a real piano does not sound like a sum of perfect harmonics.

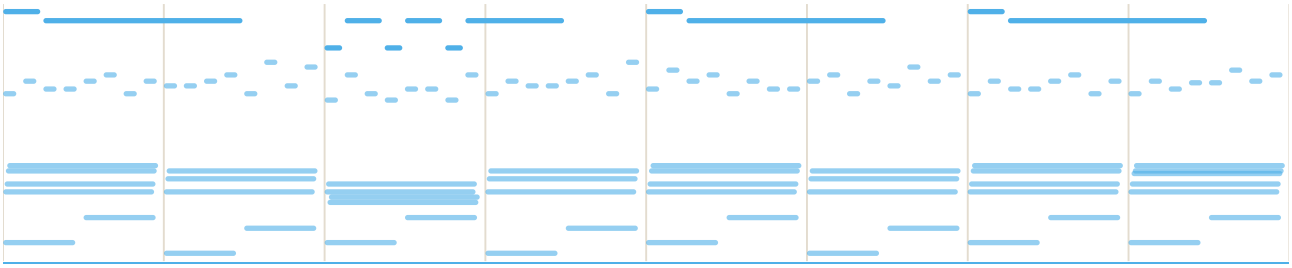
Sampling plays back recordings of real instruments. It sounds the most realistic, but needs a recording for each note (or close to it) and becomes unnatural when a recording is stretched far from its original pitch. Lastly, the Web Audio API is a W3C standard that gives web pages a graph of audio nodes (oscillators, filters, delays, convolution reverbs, compressors and waveshapers) running in the browser's own audio thread (W3C, 2021). It is the only

technology in this project's stack that produces sound, which made understanding its limits on performance especially important.

2.6 Gaps in Research and Justification for This Project

Despite the long history of algorithmic composition, several gaps exist which act as justification for this project. The first is structure: many generators produce music that follows the rules note by note but lacks themes that return, sections that build, or parts that react to each other. The second is editability: neural systems produce audio that cannot be changed, and many rule based systems are research prototypes without an interface. The third is sound: generated music is very often played through basic synthesisers or General MIDI, which makes even good notes sound cheap. The fourth is reproducibility and permanence: a game and its players need a song to stay the same once it has been chosen, even as the generator behind it improves.

By building on all of these gaps, this project aims to create a system that writes structured songs with real musical ideas, plays them convincingly, lets anyone change them, and keeps them exactly as they were saved.



Bars 25 to 32 of "riveted-hearth" in the Spirited mood, as the composer writes them

3 Design

3.1 Overview of design goals

This project aimed to design a music generator that could be dropped into a game and left to write that game's soundtrack, while also being a tool that people enjoy using on its own. The design principles were guided by the research in Chapter 2 and by the practical needs of Squimble Quest. The main goals were as follows:

- **No AI, only rules.** Every choice comes from music theory and seeded random numbers, so every note can be explained and changed.
- **The same seed, the same song.** A song is a small set of settings, and the notes are written from those settings every time it plays. The same settings must always produce exactly the same notes, on any computer.
- **A composer with no page and no audio.** The composer is plain mathematics, so that it can be tested in Node.js and loaded by the game as it is.
- **A band, not a MIDI file.** Instruments should sound like players in a room, with releases, bodies, ornaments and a shared reverb.
- **Saved songs never change.** Improving the composer must never change a song that someone has already saved.
- **Usable by anyone.** A person with no musical training should be able to make a song they like in a few clicks, while a musician can change any note.

From the perspective of game design, the generator also had to fit the way games use music. A game needs calm music for exploring, music for towns that loops seamlessly, music for cutscenes that builds, and music for fights that drives. These became the seven **styles** discussed in section 3.3, which can be combined with any of the sixty two **moods**.

3.2 System structure

3.2.1 Overall system architecture

The project is split into a small number of files, each with one job. The most important design decision was to separate the composer, which turns settings into notes, from the band, which turns notes into sound. The composer (`music.js`) never touches the page or the Web Audio API. It reads two files of plain data, `instruments.js` (every sound the band can play) and `moods.js` (every mood and flavour), and produces a list of notes with times in beats. The band (`synth.js`) takes those notes and plays them. The page (`maker.js`) sits on top of both, and is the only part that the game will not need. Figure 3 outlines how the files relate to each other.

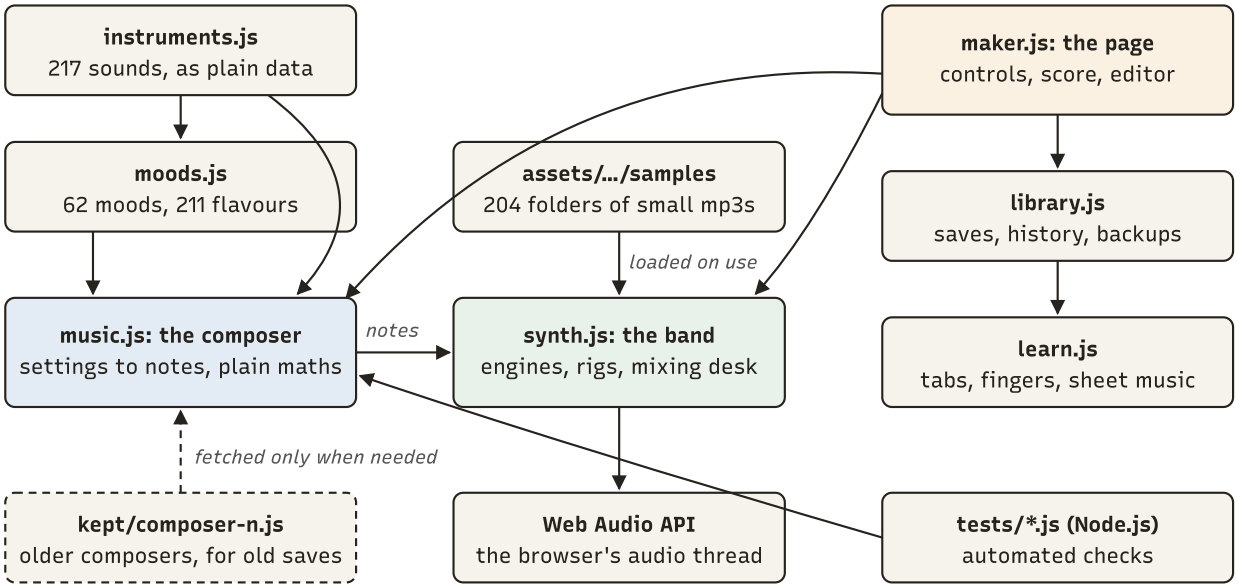


Figure 3 A system diagram outlining the files of the music maker and how they depend on each other

The load order matters. The page loads `instruments.js`, then `synth.js`, `moods.js` and `music.js`, then the rest. Since `synth.js` does not create any audio until the first note plays, Node.js can load it too, which lets the tests check the band's data without a browser. The older composers in `kept/` are only fetched when a song made by one of them is opened, which saved every visitor from downloading 1.85 MB of code they would almost never need.

3.2.2 Technology stack

- **JavaScript (vanilla)** – All of the project is written in plain JavaScript with no framework and no build step. This keeps the files readable as they are and means the game can load the composer with a script tag.
- **Web Audio API** – Used for all sound: oscillators with custom waves, buffers for plucked and struck notes, filters, convolution reverb, waveshapers for

distortion and tape, compressors and a limiter. Songs can also be rendered to a WAV file offline with an `OfflineAudioContext`.

- **HTML and CSS** – Used for the page and its controls. The score, the note editor and the learn tab are drawn on canvases.
- **IndexedDB and localStorage** – Used to keep saved songs, their history, a bin and backups in the browser, with no server and no account.
- **CompressionStream** – The browser's built in deflate is used to compress saved notes and share links.
- **lamejs** – A JavaScript port of the LAME MP3 encoder, fetched only when someone downloads an MP3.
- **Node.js** – Runs the automated checks, and the tools that build the sample library (with git and ffmpeg) and keep old composers.

3.3 The song model

A key design choice is that a song is not stored as notes. A song is a small object of settings: its mood, style, seed, key, scale, tempo, chord progressions written as roman numerals, the band (each layer with a role, an instrument, a pattern and its own seed), the sections, and the room it is played in. The notes are written from these settings by `composeSong()` every time the song plays. Figure 4 shows the start of the settings for the song in figure 1.

```
{ "name": "Mossy Lantern", "seed": "mossy-lantern", "composer": 5, "mood": "nostalgic", "style":
"tune",
  "key": 11, "scale": "lydian", "tempo": 93, "meter": "4/4", "swing": 0, "humanise": 0.507,
"colour": 0.386,
  "chordBars": 1,
  "parts": { "A": "I - V -", "B": "Imaj7 IVmaj7", "C": "I V #iv° V" },
  "layers": [
    { "id": "melody", "role": "melody", "instrument": "soft-lead", "pattern": "fragments",
"seed": "7z30w9",
      "volume": 0.674, "pan": -0.02, "octave": -1, "density": 0.699, "rate": "medium", "reverb":
0.427, ... },
    { "id": "counter", "role": "counter", "instrument": "e-piano", "pattern": "counterpoint",
"seed": "dsbbol", ... },
    // keys: wurlitzer (broken chords), pad: strings (sustain), bass: moog-bass (held)
  ],
  "sections": [ /* A, B, Solo, A, C: eight bars each */ ],
  "fx": { "reverb": 0.45, "echo": 0.18, ... } }
```

Figure 4 The settings generated for the seed "mossy-lantern" in the Nostalgic mood and Theme style (shortened)

This design has several benefits. The settings are small enough to share in a link, and they can be changed one at a time: changing the reverb does not change any notes, and writing new chords for part B does not change part A. It also means that a remix can run a single pick again with a new seed, leaving everything else alone. However, it also creates the problem that any change to

the composer changes the notes of every saved song, which is addressed in section 4.11.

A song is made by `generateSong(mood, style, seed, mods, flavour)`, which runs a series of picks in a fixed order. Each pick has its own stream of random numbers (section 4.1), so that remixing one leaves the others alone. The song is then turned into notes by `composeSong(song)`, which writes each layer's part and then has the band listen, ornament, pedal, change key and add feel. Figure 5 displays this process.

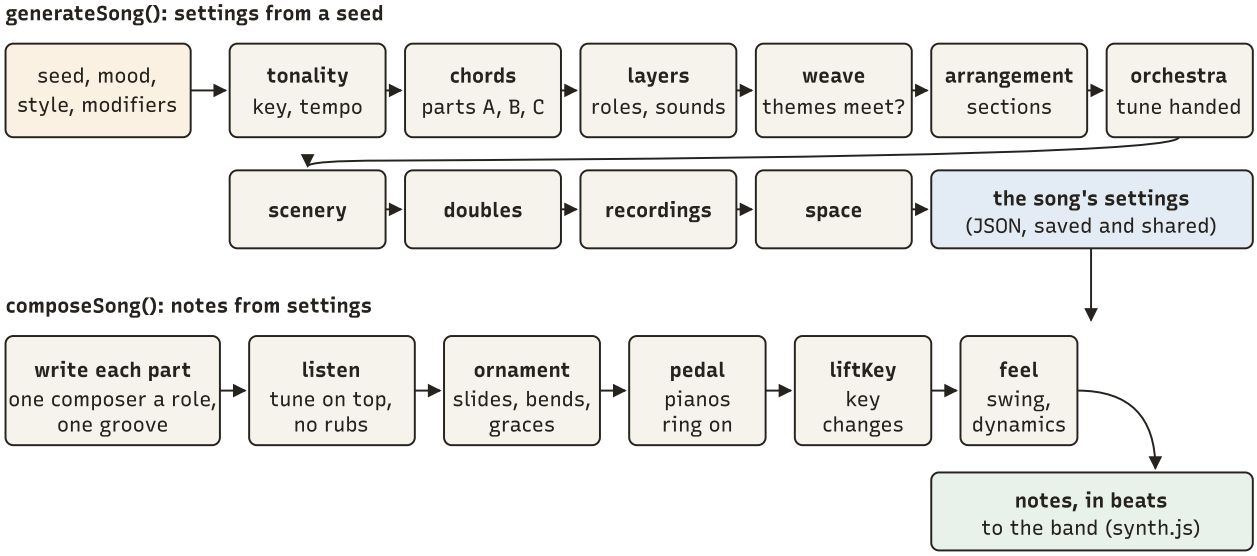


Figure 5 The order of the picks that write a song's settings, and the steps that turn those settings into notes

Every time in the composer is measured in beats from the start of the song rather than seconds, so changing the tempo never changes the notes. Only the band converts beats into seconds. Likewise, the moods are data rather than code: a mood is an entry in `MOODS` listing its scales, tempos, chord habits, which instruments it can choose for each role, how likely it is to have drums, how harsh its instruments are allowed to be (`maxEdge`), and so on. A **flavour** is a kind of music within a mood (the Underwater mood holds both a dreamy deep and a calypso party on a reef), and a **modifier** is a word in front of a mood (calm, angry, jazzy, lo-fi, epic, rising) that changes how its band plays. The seven **styles** are named after what a game needs: Explore, Theme, Town (which loops with no intro or outro), Cutscene (which always builds), Skirmish, Boss fight and Final boss (two phases, the second a key higher).

3.4 User interface design

The user interface was designed with the goal of being approachable for someone with no musical knowledge, while still giving full control to someone who has it. These two goals pull in different directions, and the interface has gone through several rounds of changes trying to balance them. A priority has

been put into showing the music itself, so the first thing under the controls is the **score**: every layer of the band drawn as coloured bars across the song's sections, with the chords written along the top, as shown in figure 6.

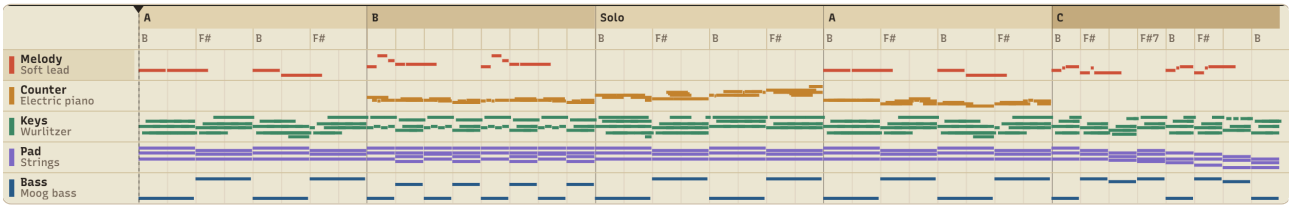


Figure 6 The score, showing each layer's notes across the song's sections, with chord names above

Each role has its own colour that is used everywhere it appears (the melody is always red and the bass always blue), so the score, the band tab, the arrange grid and the note editor all read as one system. This follows Nielsen's heuristic of "consistency and standards" (Nielsen, 1995). Under the score are the controls that most people will use most of the time: the mood (grouped into ten families so that sixty two moods are not shown at once), its flavours, the style, the modifiers and the remix buttons, as shown in figure 7.

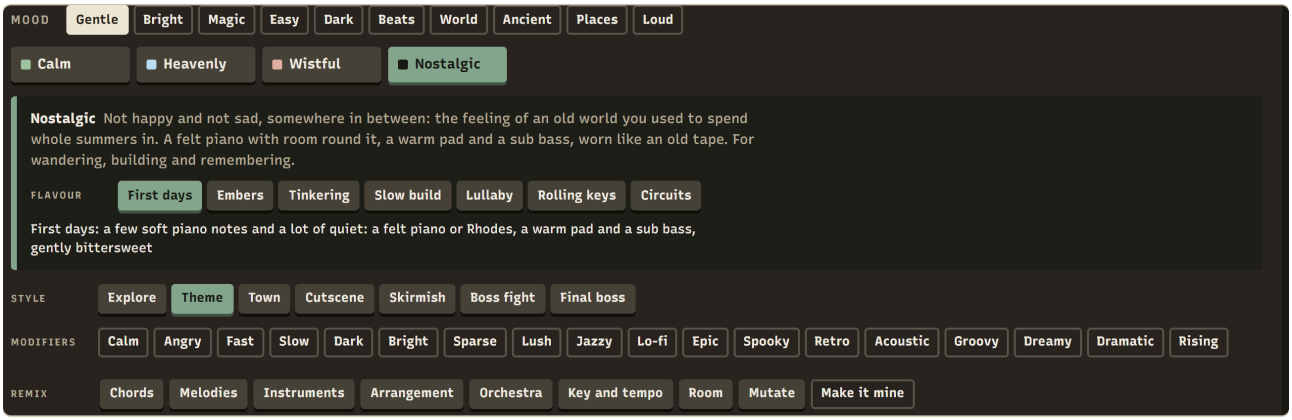


Figure 7 The mood, flavour, style, modifier and remix controls

Every mood and flavour has a short description that describes the feeling it is meant to give and where in a game it might be used, rather than any technical detail. The page deliberately never names the game, film or band that a mood was inspired by; it describes the feeling instead, so that the music stands on its own. Every control also has a tip, shown in a line docked to the bottom of the window when the pointer is over it (or after a tap on a phone). This follows the heuristic of "recognition rather than recall": a user never needs to remember what "Orchestra" does, since the page tells them as they point at it.

The more detailed controls are kept in five tabs: Song (key, scale, tempo, time signature and chords), Band (each layer's instrument, pattern and mix, and a note editor), Arrange (which layer plays which part in which section), Room (reverb, echo, tape and amp) and Learn. This follows the idea of progressive disclosure, showing simple controls first and detailed ones on request (Nielsen, 2006).



Figure 8 The Song tab, with chords written as roman numerals for parts A, B and C, and the chords they make in this key below

Chords are written as roman numerals (`I V vi IV`) rather than letter names, so they work in any key and changing the key moves the whole song. A user can type their own progression, and the page shows the chords it makes underneath and points out anything it could not read. The Band tab, shown in figure 9, lets a user pick any layer's instrument, how it plays, its mix, and edit its notes on a piano roll.

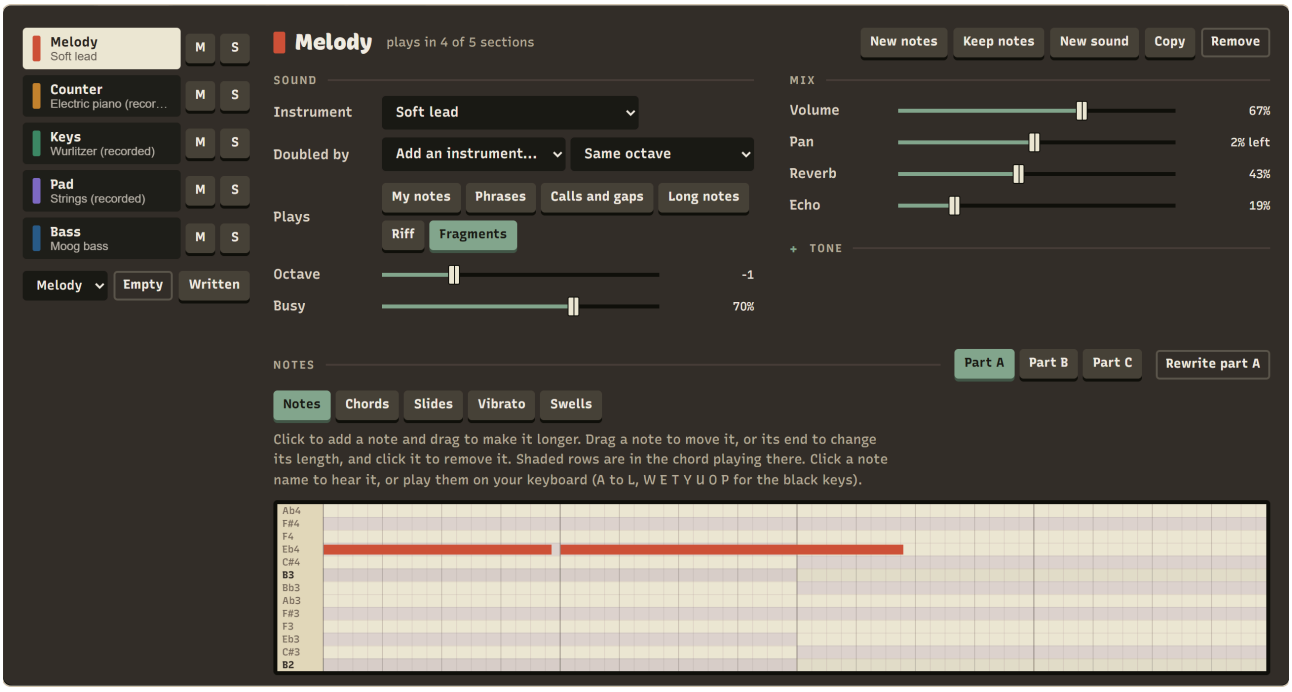


Figure 9 The Band tab, showing the melody layer's sound, pattern, mix and note editor

The Arrange tab (figure 10) was redesigned after I found it frustrating to make songs by hand. It works like a tracker, where each square says which part (A to

H) a layer plays in a section. This means the drums can keep part A's beat under part B's melody, and a single section can be given its own version of a part without affecting the others.

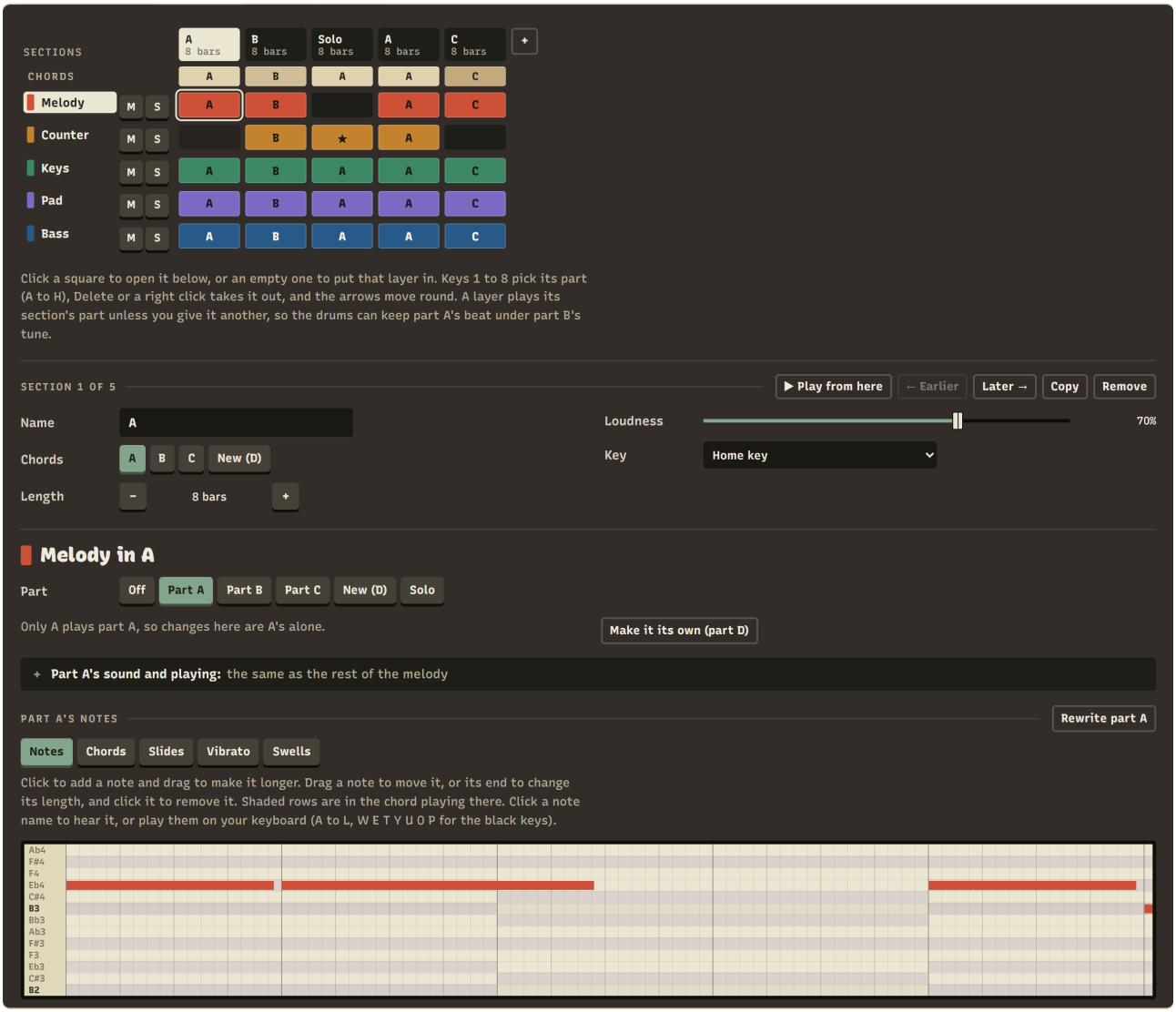


Figure 10 The Arrange tab: a grid of which part each layer plays in each section, with the picked section's settings and note editor below

The interface avoids emojis entirely, using small line icons and plain words, which keeps it consistent with the rest of my website and readable for screen readers. The canvases carry text descriptions for screen readers, the keyboard plays the open layer (A to L for white keys, W E T Y U O P for black keys), Space plays and stops, and Ctrl + Z undoes any change.

After friends tried the page on their phones and found it cluttered, the layout was reworked. On a phone, the controls are edge to edge, the long rows of moods, flavours and modifiers become single lines that can be swiped, Play becomes a full width button, and a short welcome card explains the page in three steps on the first visit. Figure 11 shows the page on a phone sized screen.

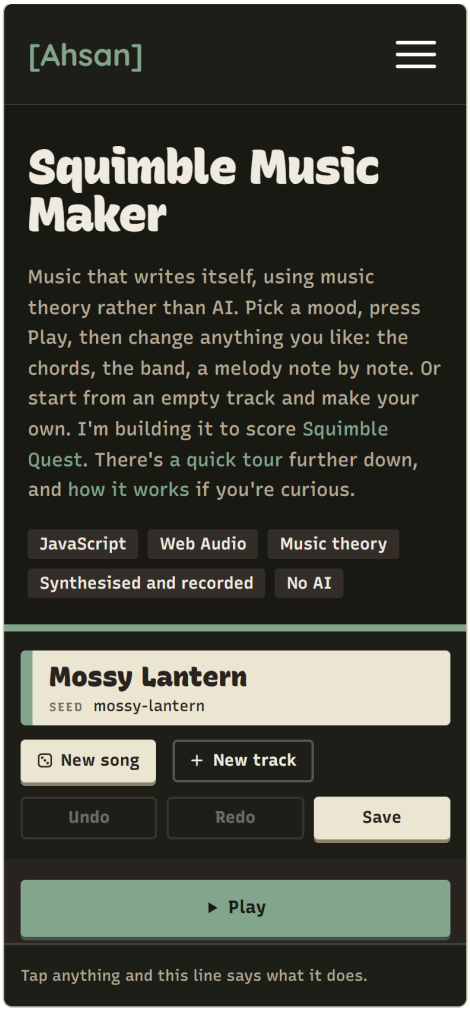


Figure 11 The music maker on a phone sized screen (390 pixels wide)

3.5 Design iterations and improvements

Throughout development, the music maker went through many rounds of revisions in response to songs that sounded wrong. A rule that I followed was that every fix gets written down, along with what it sounded like before, so that nobody (including me) undoes a fix by accident. Most of these fixes also became automated checks. This section outlines the most significant changes and the reasoning behind them, and figure 12 lists a selection of others.

3.5.1 Drums out of time

In the first version, the drums chose whether to hit on each step by rolling dice, and the kick drum, bass and chord stabs each had their own rhythm. The result sounded like three people who had never met playing at once. To resolve this, a single **groove** was introduced: a bar written out by hand like a drum machine chart, which the drums, a syncopated bass and the stabs all read, so that the kick and bass land together (section 4.5).

3.5.2 Sounding like MIDI

Early songs had reasonable notes but sounded cheap. The causes were a long list of small things: notes that stopped dead with no release, every note equally bright however hard it was played, mono pads, a reverb with no walls, no instrument bodies and no ornaments. Each of these was addressed separately: releases, brightness that follows how hard a note is played, scooped blown notes, wide pads, early reflections, modal synthesis for struck instruments, resonant bodies, slow pitch drift, slides and swells. Later, a mixing desk was added with one shared room for every instrument, and recordings of real instruments were introduced and picked five times as often as synthesisers.

3.5.3 Blends replaced by modifiers

Originally a song could blend two moods, mixing a second mood's band into the first. In practice, "folk blended with angry" sounded like two bands playing in neighbouring rooms rather than an angry folk song. Blends were replaced by modifiers, which change how the mood's own band plays (faster, harder, darker, with more swing) instead of who is in it.

3.5.4 Forgettable melodies

The first melodies were random walks over the scale. They were never wrong, but no one could remember them after the song ended. The melody writer was rewritten to grow each tune from a single motif, repeated, moved and answered in question and answer phrases, with a second idea for contrast and an a a b a shape in longer sections (section 4.3). This aligns with Huron's (2006) view that repetition is what makes music predictable enough to enjoy.

3.5.5 Saved songs changing

Since songs are stored as settings and the notes are written fresh each time, every improvement to the composer changed every song that people had saved. A song someone loved could become a different song overnight. This was solved by keeping each older composer and stamping every song with the composer that made it (section 4.11).

3.5.6 Guitar chords no hand could play

When the Learn tab was added, it revealed that many guitar chords were impossible to play: the composer stacked a guitar's notes the way it would a piano's, so some chords needed five fingers or one finger on several frets at once. A model of a guitarist's hand was written (section 4.10), and every chord given to a guitar is now one that a hand can hold.

What it sounded like	Why	Fix
Parts smearing apart	Each layer had its own timing nudge	One shared "pocket" for the band
6/8 sounding lopsided	Swing bent its triplets	No swing in 6/8
Ghost drum hits off the beat	Drums went through the dotted echo	No echo on drums or bass

What it sounded like	Why	Fix
Seasick pianos and bells	Vibrato and detune on everything	None on struck or plucked instruments
Bells clashing with chords	Inharmonic FM ratios	Harmonic ratios for gentle bells
Layers blinking in and out	Random wobble on each section's energy	Once a layer is in, it stays in
The band pumping on every kick	Drums at full level drove the compressor	Drums at 0.7, their volume set by energy
Drums out of key	Kicks, toms and ticks at fixed pitches	Tuned drums (`tuneDrum`)
Arp over the tune, sevenths grinding against it	Nobody knew what the melody played	The band listens (`listen`)
Organ, tuba and fiddle never chosen	0.6 × 0.75 is 0.4499… in floating point, just under the cap	A tolerance of 10^{-9} on the cap
Game music winding down	Tunes had intros, outros and a home chord	The Town style, which loops
Voices robotic on a moving tune	One recording bent through a whole phrase	Each note sung on its nearest recording, moving like a singer
Songs stuttering on a computer	A 10 ms audio buffer, and distortion curves turning silence into a tiny steady signal	A 40 ms buffer and odd length curves (section 5.3)

Figure 12 A selection of rows from the project's "what sounded wrong, and the fix" table

3.6 Ethical and licensing considerations

Although this project does not involve participants in the way a psychological study does, it still raised several ethical considerations that shaped its design.

3.6.1 Licensing of recordings

Every recording the band plays comes from a library whose licence allows it to be used in a game: public domain (CC0) sets, the University of Iowa's samples (free for any use), and Creative Commons Attribution recordings, which are credited by name on the page. Some sets were deliberately left out because of their licences, and the few recordings under share-alike licences are noted in the project's documentation. Where no free recording of an instrument existed, a close relative stands in (a recorder for the tin whistle, for example) or the instrument stays synthesised.

3.6.2 No AI and no training data

Since the composer uses no machine learning, no one's music was used as training data. Where songs inspired a mood, they inspired it in the way music inspires any composer: by listening to them, reading transcriptions and interviews, and writing down rules about what makes them sound the way they do.

3.6.3 Privacy

The music maker has no server, no accounts and no tracking. Saved songs live only in the user's own browser, and a backup is a file the user downloads themselves. A shared link contains only the song's settings.

3.6.4 Inclusive and accessible design

The page works with a keyboard and a screen reader, does not rely on colour alone (every role is also named), works on phones, and assumes no musical knowledge. The Learn tab is designed to help people who do not read sheet music, showing falling notes on piano keys or finger positions on a guitar neck instead.



Bars 1 to 8 of "cobwebbed-koala" in the Magic mood, as the composer writes them

4 Implementation

This chapter explains how each part of the music maker works, both technically and mathematically. The code shown in the figures is taken directly from the project, shortened in places (marked with `...`) but otherwise unchanged, comments included. The composer is in `music.js` and the band is in `synth.js` unless stated otherwise.

4.1 Random numbers and seeds

Every choice the composer makes, from the key to whether a single note gets a grace note, comes from a random number. For the same seed to always give the same song, these numbers cannot come from JavaScript's `Math.random()`, which is different every time and cannot be seeded. Instead, the project uses its own seeded random number generator, built from two small, well known algorithms.

The first is the FNV-1a hash (Fowler et al., 2019), which turns any text into a 32 bit number. It starts from a fixed "offset basis" and, for each character, combines it in with an exclusive or and then multiplies by a fixed prime, keeping only the bottom 32 bits:

$$h_0 = 2166136261, \quad h_i = (h_{i-1} \oplus c_i) \times 16777619 \mod 2^{32} \tag{1}$$

The second is mulberry32, a tiny generator with a single 32 bit state that is advanced by a constant and then scrambled with shifts and multiplications. It is not suitable for cryptography, but it is fast, passes common statistical tests and, most importantly, gives the same sequence for the same starting state on every computer. Figure 13 shows both.

```
// a string to a 32 bit number (fnv-1a), so any words can be a seed
function hashText(text) {
  let hash = 2166136261;
  for (const char of String(text)) {
    hash ^= char.codePointAt(0);
    hash = Math.imul(hash, 16777619);
  }
  return hash >>> 0;
}

// a seeded random number maker (mulberry32). the parts are joined into its seed, so
// makeRng(song.seed, layer.seed, 'A') is the same numbers every time
function makeRng(...parts) {
  let state = hashText(parts.join('|'));
  const next = () => {
    state = (state + 0x6d2b79f5) | 0;
    let t = Math.imul(state ^ (state >>> 15), 1 | state);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
  return {
    next,
    range: (low, high) => low + next() * (high - low),
    int: (low, high) => low + Math.floor(next() * (high - low + 1)),
    chance: (p) => next() < p,
    pick: (list) => list[Math.floor(next() * list.length)],
    ...
  };
}
```

Figure 13 hashText() and makeRng(): the seeded random numbers behind every choice the composer makes

The important detail is that `makeRng` takes several parts and joins them into its seed. The key is picked from `makeRng(seed, mood, 'tonality')`, the chords from `makeRng(seed, mood, 'chords')`, and so on. Each pick therefore has its own independent stream of numbers. If the chords were picked from the same stream as everything after them, then writing new chords would use up a different amount of random numbers and change every later choice too. With separate streams, the "Chords" remix button gives the chords a new nonce and leaves the band, melody and arrangement exactly as they were. The same idea goes all the way down: each layer has its own seed, and an ornament's dice come from the note's place in the bar, so a motif that returns is ornamented the same way each time.

Seeds are two words, which also become the song's name. Since Squimble Quest is a child's dream, the words come from every world a dream could wander into: forests, deserts, snowy mountains, pirate ships, haunted houses and the future. There are 679 first words and 777 second words, which gives 679 × 777 = **527,583** possible seeds, from "mossy lantern" to "burpy" and "peglegged" combinations. Because the seed is hashed as text rather than looked up by its position in the list, adding new words never changes the song of an existing seed.

4.2 Harmony

4.2.1 Scales and chords

A scale is stored as the number of semitones each of its notes sits above the key. The major scale is `[0, 2, 4, 5, 7, 9, 11]`, lydian raises the fourth to `6`, and harmonic minor is `[0, 2, 3, 5, 7, 8, 11]`. There are twenty scales in total, including the blues scale, the Japanese hirajoshi and in sen scales, the double harmonic scale (raga bhairav) and approximations of the two tunings of a Javanese gamelan, pelog and slendro. Chords are built by stacking every other note of the scale. If s is the list of steps with n notes, the note at scale position i (which can go past the octave) and the triad on degree d are:

$$\text{note}(i) = s[i \bmod n] + 12\lfloor i/n \rfloor, \quad \text{chord}(d) = \{\text{note}(d), \text{note}(d+2), \text{note}(d+4)\} \tag{2}$$

This is why a dorian song automatically gets dorian chords. Scales with fewer than seven notes (pentatonic scales) say which seven note scale their chords should come from, since stacking every other note of a five note scale does not make proper triads. Chords are written as roman numerals, which the parser turns into a root (semitones above the key) and a list of tones above the root: upper case is major (`[0, 4, 7]`), lower case is minor (`[0, 3, 7]`), a flat or sharp moves the root by a semitone, and suffixes such as `maj7`, `sus4` or `°` change the tones. A song's **colour** setting then adds sevenths with probability 0.85 × colour, ninths with 0.45 × colour and suspensions with 0.12 × colour, so a colourful song gets jazzier chords.

4.2.2 Writing progressions

Each of a song's three parts (A, B and C) gets a chord progression written in one of four ways, chosen by weight: a **bank** of progressions that suit the mood (weight 5), a **Markov** walk (weight 3), a two chord **vamp** (weight 1.2), and, for horror moods, **chromatic** roots that wander by odd intervals. Parts B and C are tried up to six times to make sure they differ from A. The Markov walk uses the table `CHORD_MOVES`, which says how often each chord of the scale leads to each other one. Turning each row into probabilities gives:

$$P(\text{next} = j \mid \text{current} = i) = \frac{w_{ij}}{\sum_k w_{ik}} \tag{3}$$

Figure 14 shows these probabilities. The table encodes the "tonic, predominant, dominant, home" pull described by Piston and DeVoto (1987): V goes home to I half the time, ii leads to V half the time, and vii° resolves to I more than two times in three.

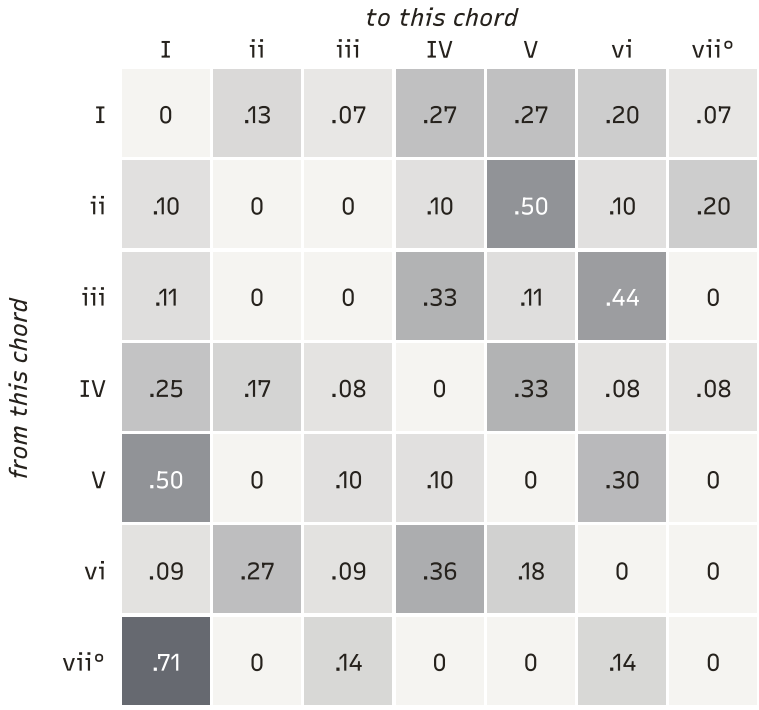


Figure 14 CHORD_MOVES as probabilities: how likely each chord of the scale is to lead to each other chord in a Markov progression

Progressions with notes outside the scale are made less likely, and when a modifier shades a song into another scale (making it darker, for example), its progressions are moved with it by degree, so `I V vi IV` in major becomes `i v bVI iv` in minor (`reharmonise`).

4.2.3 Voice leading

Once the chords are known, each one needs to be voiced: which octave each note goes in, and which note is at the bottom. As discussed in section 2.4, good voice leading moves each note as little as possible from the last chord. `voiceChord()` tries every inversion of the chord in every nearby octave and gives each a cost. If p is the previous chord's notes, v a candidate voicing, c the centre of the role's range and λ a weight (0.3 if there is a previous chord, otherwise 1, with no movement term), the cost is:

$$\text{cost}(v) = \lambda \cdot \left| \frac{v_{\text{low}} + v_{\text{high}}}{2} - c \right| + \Sigma_i \left| v_i - p_i \right| \tag{4}$$

The voicing with the lowest cost wins. The first term stops the chords from slowly creeping up or down out of the instrument's range, and the second keeps each voice moving by small steps. This is a direct, if simple, application of Tymoczko's (2011) idea of voice leading as the shortest distance between chords.

```
// voices a chord around a note: every inversion in every nearby octave, keeping whichever moves
// least from the last chord (voice leading), or sits nearest the centre if there wasn't one
function voiceChord(rootPc, tones, center, previous) {
  let best = null;
  let bestCost = Infinity;
  for (let inversion = 0; inversion < tones.length; inversion++) {
    const shape = tones.map((tone, i) => tone + (i < inversion ? 12 : 0)).sort((a, b) => a - b);
    const base = center - 6 + mod12(rootPc - (center - 6)) - 12;
    for (let octave = -1; octave <= 2; octave++) {
      const notes = shape.map((tone) => base + tone + 12 * octave);
      const middle = (notes[0] + notes[notes.length - 1]) / 2;
      let cost = Math.abs(middle - center) * (previous ? 0.3 : 1);
      if (previous) for (let i = 0; i < Math.min(notes.length, previous.length); i++) cost +=
Math.abs(notes[i] - previous[i]);
      if (cost < bestCost) {
        bestCost = cost;
        best = notes;
      }
    }
  }
  return best;
}
```

Figure 15 voiceChord(): choosing the inversion and octave of a chord that moves least from the one before

4.3 Melody

As discussed in section 3.5.4, melodies are grown from a single **motif** rather than chosen note by note. A motif is a rhythm (a list of note lengths that fills part of a bar, picked from cells of slow, medium and fast notes to match the layer's busyness) and a **contour**, a shape that says how many scale steps each note sits above or below the first. There are fourteen contours, each a small function of the note's position i and the number of notes n. For example, an arch rises and falls with a sine curve, $a(i) = \text{round}(h \cdot \sin(\pi i / n))$, and a "leap" jumps up three to five steps and then steps back down into the gap it left, which is one of the oldest rules of melody writing. Figure 16 draws all fourteen.



Figure 16 The fourteen melodic contours in CONTOURS, each drawn for eight notes (scale steps above or below the first note)

Ambient songs favour arches, turns and pendulums, which drift and circle, while tunes favour leaps, arches and repeated notes, which are easier to remember. Flavours can ask for their own shapes: a march prefers the bugle

call shape ("triad"), a lament the falling pairs of a "sigh", and a sea shanty the rolling "wave".

The motif is then built into phrases. A phrase is usually four bars long, and phrases alternate as a **question** and an **answer**: the question might play the motif, then the motif moved up a step (a sequence), and end on an unstable note, while the answer brings back the motif and ends on a stable one. In a long section, the third phrase of every four is a contrasting middle built on a **second idea** with its own rhythm, giving the a a b a shape found in a great many songs. On top of this:

- Notes on strong beats snap to the nearest chord tone, in the direction the shape was heading, so the melody always fits the harmony where it matters most.
- Part B either raises the motif by two or three scale steps or gives it a new rhythm, and part C usually slows it to under half its busyness, so each part sounds related but different.
- Later times round a part can change how its last phrase ends, so a theme coming back for the third time is not a copy.
- No line plays more than six notes a second (`MAX_RATE`), there are no rests inside a bar (melodies breathe at the ends of phrases instead), and every melody uses at least two lengths of note.
- Solos are built four bars at a time from new ideas, never repeat themselves, climb to a peak, and land on the root of the chord.

4.4 Counterpoint

A counter melody set to "second melody" writes a line that fits the melody note for note, the way Toby Fox stacks two tunes. It follows the rules of counterpoint from section 2.4, written as a cost function. For each note of its rhythm, it tries every pitch in its range that is in the scale or the chord, and keeps the cheapest. If g is the interval in semitones from the candidate note up to the melody, the cost adds up:

- 3 if the beat is strong and the note is not in the chord;
- 12 if the note is above the melody (it must stay underneath), and 5 if it is within two semitones under it;
- the interval's dissonance from `CONSONANCE` (shown in figure 17), halved on a weak beat, where a clash moving by step is a passing note;
- 0.4 for moving the same way as the melody, and −0.5 (a reward) for contrary motion;
- 4 for parallel fifths or octaves (the same perfect interval twice in a row while both voices move);
- 0 for a step, 0.4 for a third, 1.5 for a leap up to a fifth, 4 for anything larger, and 0.7 for repeating a note.

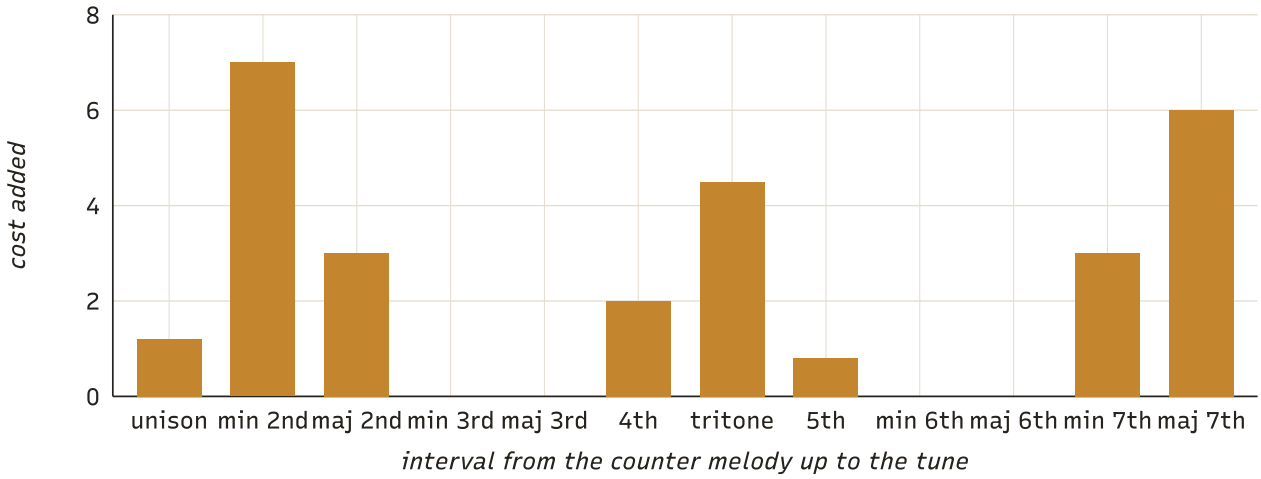


Figure 17 CONSONANCE: the cost of each interval between the counter melody and the tune above it. Thirds and sixths are free, while seconds, sevenths and the tritone are expensive

A tiny tie breaker based on the pitch and the position in the bar, `((midi × 7 + round(t × 12) × 13) mod 10) / 33`, makes sure two equally good notes are always decided the same way, so the same bar always gets the same line. The counter melody also has its own slow rhythm that it switches to in bars where the melody is busy, so it holds where the tune moves and moves where the tune holds.

```
  for (let midi = from; midi <= to; midi++) {
    const pc = mod12(midi);
    if (!c.scalePcs.has(pc) && !chord.pcs.has(pc)) continue;
    let cost = strong && !chord.pcs.has(pc) ? 3 : 0;
    if (tune) {
      const gap = tune.midi - midi;
      if (gap < 0) cost += 12;
      else if (gap > 0 && gap < 3) cost += 5;
      if (gap > 19) cost += 1.5;
      // a clash on a weak beat, moving by step, is a passing note, and half as bad
      cost += CONSONANCE[mod12(gap)] * (strong ? 1 : 0.5);
      const move = last ? Math.sign(midi - last.midi) : 0;
      if (tuneMove && move) cost += move === tuneMove ? 0.4 : -0.5;
      if (last?.tune && move && tuneMove) {
        const before = mod12(last.tune.midi - last.midi);
        if ((before === 0 || before === 7) && before === mod12(gap)) cost += 4;
      }
    }
    if (last) {
      const leap = Math.abs(midi - last.midi);
      cost += leap === 0 ? 0.7 : leap <= 2 ? 0 : leap <= 4 ? 0.4 : leap <= 7 ? 1.5 : 4;
    } else cost += Math.abs(midi - c.center) * 0.15;
    cost += ((midi * 7 + Math.round(cell.t * 12) * 13) % 10) / 33;
    if (cost < bestCost) {
      bestCost = cost;
      best = midi;
    }
  }
```

Figure 18 The heart of counterpoint(): Fux's rules of counterpoint written as a cost for every candidate note

This is a greedy algorithm, since it picks the best note one at a time rather than the best line overall. A search over the whole line (like the Viterbi search in section 4.10) would find better lines in theory, but in practice the greedy version already follows the rules well, and it is fast and easy to reason about.

4.5 Rhythm, groove and feel

As described in section 3.5.1, the drums, a syncopated bass and the chord stabs all read one **groove**: a bar written out by hand as text, one character per sixteenth note (or eighth note in 6/8), like a drum machine chart. Figure 19 shows two of them. There are twenty six kinds of groove in 4/4 alone, from a plain rock beat to brushes on a jazz ride, half time, a march, a bossa nova, a reggae one drop, a boom bap beat, trap and a Middle Eastern maqsum.

```
// a groove is a bar written out by hand like a drum machine chart:
//   k kick, s snare, h hat, o open hat, p perc (rim, shaker, tick, or a rock kit's ride), l low
perc
//   (toms, taiko), y crash, c comping: where stabs, a syncopated bass and power chords go
// x a hit, - a soft (ghost) one, . nothing, a character a step (16ths, or 8ths in 6/8)
const GROOVES = {
  '4/4': {
    groove: [
      { k: 'x.......x.......', s: '....x.......x...', h: 'x.x.x.x.x.x.x.x.', c:
'x.....x.....x...' },
      { k: 'x..x....x.x.....', s: '....x..-.-..x...', h: 'x-x-x-x-x-x-x-x-', c:
'..x...x...x...x.' },
      ...
    ],
    // brushes: a jazz ride (ding, ding-a ding) with the brush swishing on 2 and 4, for cozy and
    // wistful songs. it sounds best swung
    brush: [
      { k: 'x.......-.......', s: '....x.......x...', h: 'x...x.x.x...x.x.', c:
'x.....x.........' },
      ...
```

Figure 19 Part of GROOVES: drum charts written as text, read by the drums, the bass and the chord stabs together

Writing notes perfectly on the grid sounds mechanical, so `feel()` adds swing, humanising and dynamics after everything else is written. **Swing** delays every off beat. If a note sits at position u within its beat (0 to 1) and the song's swing is s, the off beat at 0.5 is moved to $m = 0.5 + s/4$ and the rest of the beat is stretched around it:

$$u' = u \,/\, 0.5 \cdot m \quad \text{if } u < 0.5, \qquad u' = m + (u - 0.5) \,/\, 0.5 \cdot (1 - m) \quad \text{otherwise} \tag{5}$$

At $s = 2/3$ the off beat lands at 2/3 of the beat, which is exactly a triplet swing, the feel of most jazz. Figure 20 plots this mapping. Swing is never applied in 6/8, whose beats are already split into three, since it made 6/8 sound lopsided.

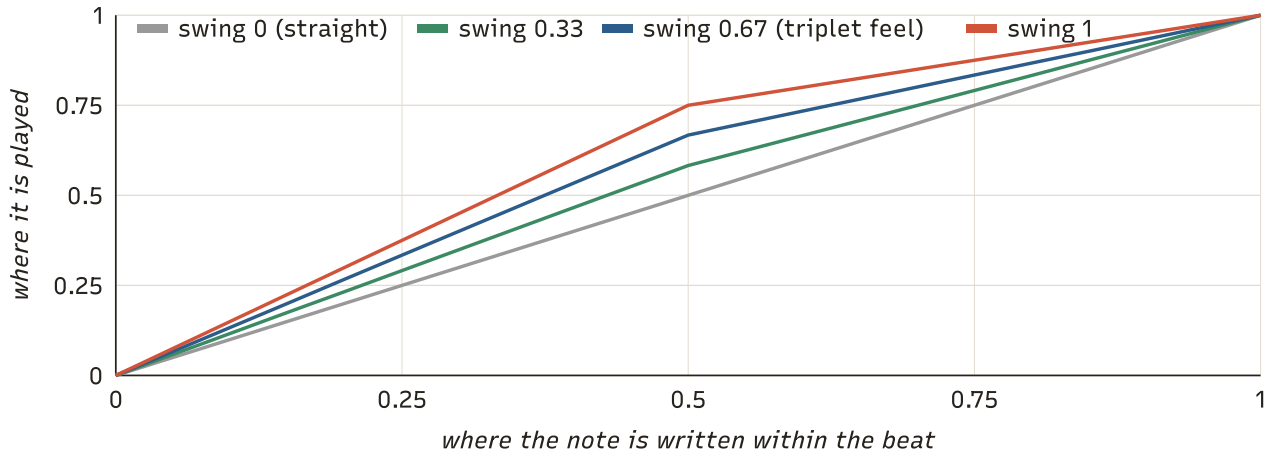


Figure 20 How feel() moves a note within its beat for different amounts of swing

Humanising is done in two parts. The whole band shares a "pocket": a random nudge of up to ±0.012 of a beat × humanise (about 6 ms at 120 bpm at full humanise), the same for every instrument at each point in time (rounded to a twelfth of a beat), so the band drifts slightly ahead and behind together the way real players do. On top of that, each role adds a little nudge of its own, set by `LOOSE`, which is almost nothing for the drums and bass so the rhythm section stays tight. Note strengths also vary by a few percent, the drums most, since an even hi hat sounds like a machine. Finally, **dynamics** scale each note's strength by its section's energy e, as $k = 0.8 + 0.27e$, and the last bar before a louder section swells by up to 12% into it.

4.6 The band listens

Parts can sound fine alone and wrong together. Early songs often had an arpeggio playing above the melody, or a pad holding a seventh that ground against a melody note a semitone away. This happened because no part knew what any other part was playing. The fix was to add a step after every part is written, `listen()`, in which the band listens to the tune and adjusts:

- Keys, arpeggios and counter melodies drop an octave if they would start above the tune (pads and bells stay where they are).
- Nothing rubs against the tune. A rub is a minor second or minor ninth against a melody note of half a beat or longer. Pads and keys drop the clashing note, and moving lines move to a chord note that does not rub or climb over the tune.
- Everything with a pitch is in key, including the tuned drums, the hum of a texture and even the crickets.
- The drummer plays softer and without ghost notes when fewer than half of the others are playing.

To find which melody note is sounding at any moment, listening uses a binary search over the melody's notes sorted by time, so each check costs $O(\log n)$ rather than $O(n)$. Whoever is carrying the tune in a section (the melody, or a counter melody that has been handed it) is treated as the tune, and is never moved by listening.

4.7 Arrangement and orchestration

A song's **arrangement** is its list of sections: which part's chords each plays, for how many bars, which layers play in it and how much energy it has. Each mood says which layers open its songs, and once a layer comes in, it stays in until a real change of gear, which fixed a problem where layers blinked in and out at random. On top of the arrangement, several techniques give a song a sense of shape:

- **Orchestration** (`passTune`): the tune is handed round the band section by section (the strings take it, then the flute), doubled by bells an octave up, and everyone plays it together the last time the theme comes round.
- **Builds**: one player starts, then everyone joins a section at a time up to a climax, followed by a breath and a finale a key higher. Cutscenes always build.
- **Weaves**: inspired by Wizard101's world themes, the tune plays alone, then a second theme alone over the same chords, and then both together. The second theme is written in counterpoint against the tune even where the tune is silent, so it is the same line alone as when they meet.
- **Key changes**: a section marked as lifted, and everything after it, plays a whole step higher (a half step a third of the time). In equal temperament, every frequency is multiplied by $2^{2/12} \approx 1.122$. The change happens after the band has been written, listened to and ornamented, so everything still fits.
- **Drifting loops**: inspired by Eno's tape loops, the bells' "drifting loops" pattern gives each of a few notes its own loop length of two to four bars plus up to two beats. In 4/4, two loops of 13 and 17 beats only line up again after lcm(13, 17) = 221 beats, about 55 bars, so the pattern keeps changing while staying calm.

4.8 Sound synthesis

The band has 217 instruments. Every recorded instrument also keeps a synthesised version, which plays until its recordings have loaded and whenever the user turns recordings off. This section explains the main synthesis engines and the mathematics behind them.

4.8.1 Waves built from harmonics

Many leads and organs are oscillators playing a custom wave, defined by the strength of each harmonic and turned into a `PeriodicWave` by the Web Audio API. Some of these waves are copied from real instruments. A Hammond organ with its drawbars pulled out to 888000000 (the classic jazz and gospel setting) has its first three harmonics loud and the rest silent. A bowed string's wave is close to a sawtooth, but with a dip around every seventh harmonic, where the bow sits on the string. The NES console's square wave channels (the sound of a lot of chiptune music) are pulse waves that are "on" for a quarter or an eighth of each cycle. By Fourier analysis, a pulse wave with duty cycle d has harmonics of strength:

$$a_n \propto \frac{\sin(\pi\, n\, d)}{n} \tag{6}$$

For $d = 1/4$, every fourth harmonic is missing (since $\sin(\pi n/4) = 0$ when n is a multiple of 4), which gives the 25% pulse its hollow, nasal sound. The code

builds the wave with exactly this formula, `Math.sin(Math.PI * (i + 1) * 0.25) / (i + 1)`, and figure 21 shows the result.

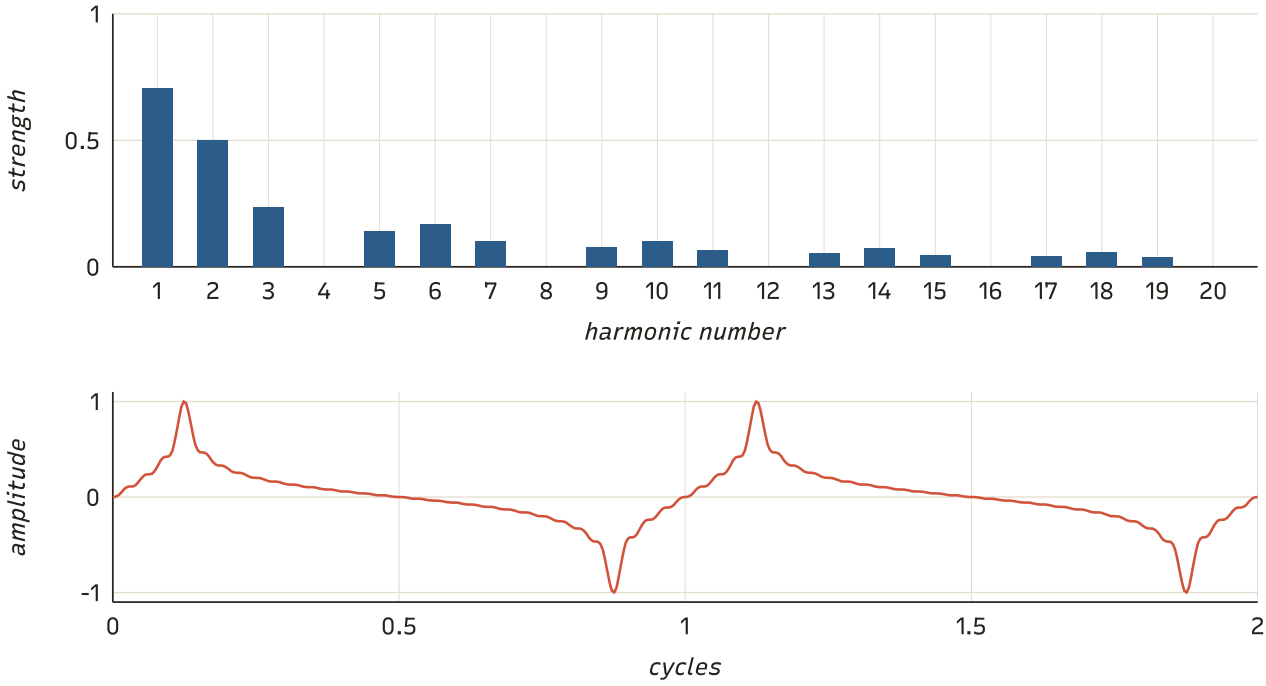


Figure 21 The NES style 25% pulse wave (WAVES.pulse25): the strength of its first twenty harmonics, with every fourth missing, and the wave they add up to

4.8.2 Plucked strings: Karplus-Strong

Plucked strings (guitars, harps, plucked basses, a sitar) use the Karplus-Strong algorithm (Karplus and Strong, 1983). A delay line of N samples, one cycle of the note long, is filled with a burst of noise. On every sample, the value leaving the delay line is averaged with its neighbour, made very slightly quieter and fed back in. The averaging is a gentle low pass filter, so the high harmonics die faster than the low ones, exactly as on a real string. With a filter weight ρ (the code calls it `ring`, 0.5 for nylon and 0.3 for a brighter steel string) and a loss g per trip:

$$y[n] = g \cdot \left((1-\rho) \cdot y[n-N] + \rho \cdot y[n-N+1] \right) \tag{7}$$

The filter itself adds a delay of ρ samples, so the loop is really $N + \rho$ samples long and the note sounds at $f' = f_s / (N + \rho)$, where f_s is the sample rate. The code therefore picks $N = \text{round}(f_s/f - \rho)$. The loss g is chosen so that the note fades by 60 dB (to a thousandth of its level) after T_{60} seconds. Since the wave goes round the loop f times a second, it makes $f \cdot T_{60}$ trips in that time, so:

$$g^{\,f\,T_{60}} = 0.001 \quad \Rightarrow \quad g = 0.001^{\,1\,/\,(f\,T_{60})} \tag{8}$$

There is one subtle problem. N has to be a whole number of samples, so the note is slightly out of tune. If N is rounded by δ samples (at most half a sample), the pitch error in cents is approximately:

$$\Delta \approx \frac{1200}{\ln 2} \cdot \frac{\delta}{N} \approx 1731 \cdot \frac{\delta}{N} \text{ cents} \tag{9}$$

For a low note, N is large and the error is tiny, but for an E5 at 44.1 kHz, N is about 66 and the error can reach 1731 × 0.5 / 66 ≈ 13 cents, which is clearly audible against a piano. The project measures this error for every note (`buffer.cents`) and the pluck engine tunes the note back by that amount when it plays it. Jaffe and Smith (1983) solve the same problem with an all-pass filter inside the loop; retuning the finished buffer was simpler here, since each note is worked out once and kept.

Two more details make the string sound like a finger rather than a pick. A real finger pulls the string into a triangle before letting go, so a `soft` amount of a smooth triangle is mixed into the starting noise, making the note start round instead of with a metallic zing. For a sitar, values past a threshold are flattened (`buzz`), imitating the string buzzing against the instrument's curved bridge. Figure 22 shows a plucked A2 made by this function, and figure 23 shows its main loop.

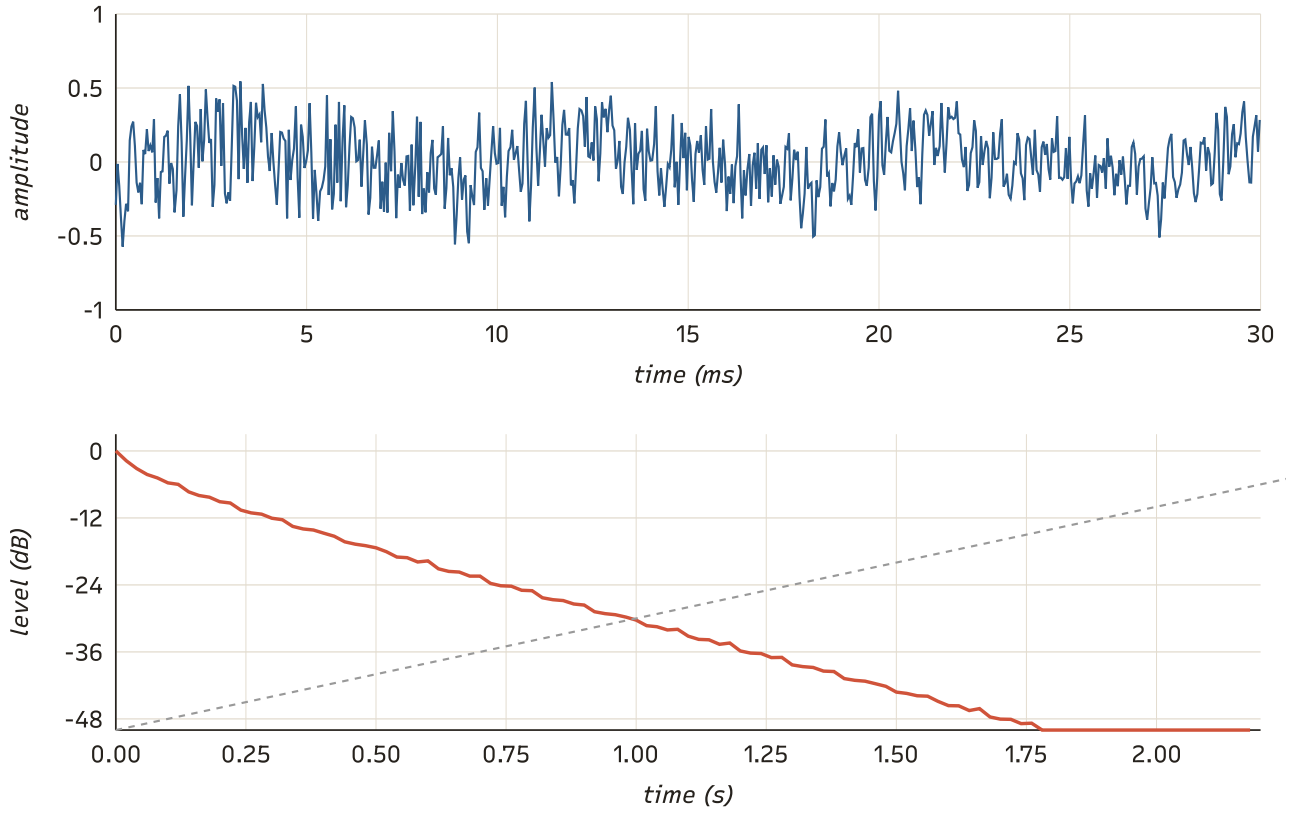


Figure 22 A plucked A2 (110 Hz) from ksBuffer(): its first 30 milliseconds, in which the same burst of softened noise comes round every 9.1 ms (one cycle of the note), and its level over two seconds against the straight line of an exact 60 dB fade in 2.5 seconds

```
const f = mtof(midi);
// the loop filter (weights 1 - ring and ring) adds ring samples of delay to the loop
const period = Math.max(2, Math.round(rate / f - ring));
...
buffer.cents = 1200 * Math.log2((f * (period + ring)) / rate);
const string = new Float32Array(period);
let last = 0;
for (let i = 0; i < period; i++) {
  last += (Math.random() * 2 - 1 - last) * bright;
  // pulled aside a third of the way along, and centred on zero
  const x = i / period;
  const pulled = (x < 0.3 ? x / 0.3 : (1 - x) / 0.7) - 0.5;
  string[i] = last * (1 - soft) + pulled * soft;
}
// what's kept each time round, so it's 60 dB quieter after decay seconds
const keep = 0.001 ** (1 / (f * decay));
const fade = Math.floor(rate * 0.3);
for (let i = 0, at = 0; i < length; i++) {
  const next = (at + 1) % period;
  data[i] = string[at] * Math.min(1, (length - i) / fade);
  let value = (string[at] * (1 - ring) + string[next] * ring) * keep;
  if (buzz && value > buzz) value = buzz + (value - buzz) * 0.25;
  string[at] = value;
  at = next;
}
```

Figure 23 ksBuffer(): Karplus-Strong string synthesis, with the tuning error measured and a finger's triangle mixed into the pluck

4.8.3 Struck things: modal synthesis

Pianos, marimbas, celestas, kalimbas, music boxes and electric pianos use modal synthesis. A struck bar or string rings at a handful of frequencies at once (its modes), each fading at its own rate, with the highest usually fading first. Each instrument lists its modes as a ratio to the note, a level, a fade time and how much harder playing brings it out. Each mode is generated by a two pole resonator, a recursive filter whose impulse response is a decaying sine wave:

$$y[n] = 2r\cos(\omega)\cdot y[n-1] - r^2\cdot y[n-2], \quad \omega = \frac{2\pi\, f_{mode}}{f_s}, \quad r = 0.001^{1/(T_{60}\, f_s)} \tag{10}$$

Starting it with $y[0] = 0$ and $y[1] = A\ r\ \sin\omega$ gives exactly $y[n] = A\ r^n\ \sin(\omega n)$, which can be checked by substituting it into the recurrence and using the identity $\sin(x+\omega) + \sin(x-\omega) = 2\sin x\cos\omega$. This means each mode costs two multiplications and a subtraction per sample, with no calls to `Math.sin` at all. The value of r follows the same reasoning as equation 8: after $T_{60}\cdot f_s$ samples the mode has faded to a thousandth.

The ratios are where the character comes from. Real marimba bars are carved underneath so that their second mode sits two octaves above the note (a ratio of about 4) and their third near 10; the project's marimba uses 1 : 4 : 9.9. A

xylophone is tuned to 1 : 3, a kalimba's tine has an inharmonic second mode near 5.9, and a piano's modes are almost whole multiples, but slightly stretched, since a stiff string's overtones are sharp. Following Fletcher (1964), mode k of a string with stiffness B sounds at $f_k = k f_0 \sqrt{(1 + B k^2)}$, which is the line `f * ratio * Math.sqrt(1 + (inst.stretch || 0) * ratio * ratio)` in the code. A piano also gets a second, slightly detuned string for each mode, which slowly beats against the first: the shimmer of a real piano's unison strings. Finally, how hard a note is struck (*level*) raises each mode by $level^{\text{hard}}$, so a loud note is brighter rather than just louder. Figure 24 shows the spectrum of a marimba note, with its three modes clearly visible.

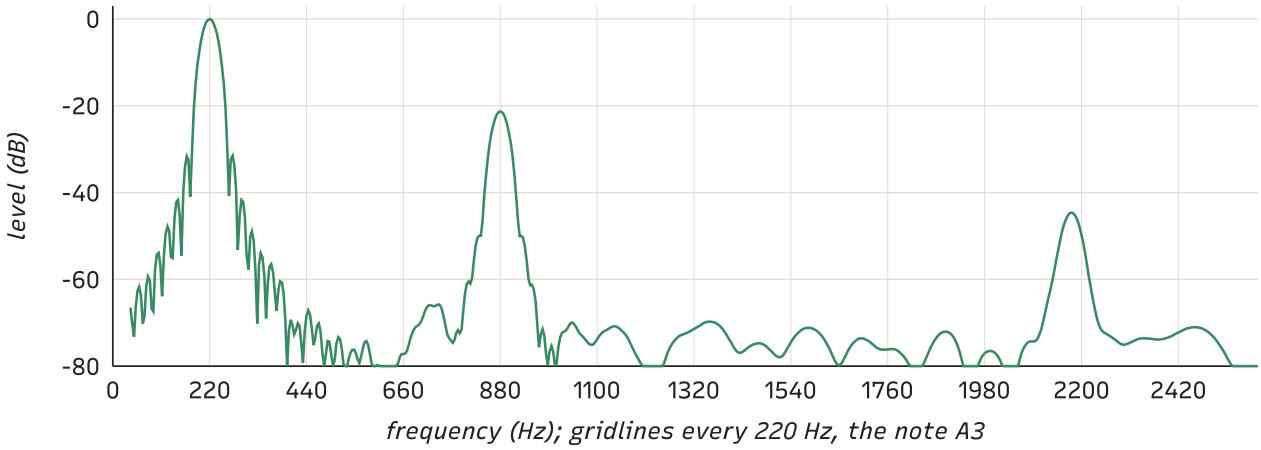


Figure 24 The spectrum of the first 46 ms of a marimba's A3 from modalBuffer(), with modes at 220 Hz, 880 Hz (four times the note) and about 2180 Hz (9.9 times, the quickest to fade), above the soft noise of the mallet

```
const ring = (hz, amount, decay) => {
  if (hz > rate * 0.45 || amount <= 0) return;
  const w = (2 * Math.PI * hz) / rate;
  const r = 0.001 ** (1 / (decay * rate));
  const a = 2 * r * Math.cos(w);
  const b = r * r;
  const end = Math.min(length, Math.floor(rate * decay * 1.2));
  let y1 = (amount / total) * r * Math.sin(w);
  let y2 = 0;
  data[1] += y1;
  for (let i = 2; i < end; i++) {
    const y = a * y1 - b * y2;
    data[i] += y;
    y2 = y1;
    y1 = y;
  }
};
const hardness = level / 2;
for (const [ratio, amount, decay, hard = 0] of inst.modes) {
  const hz = f * ratio * Math.sqrt(1 + (inst.stretch || 0) * ratio * ratio);
  ...
  ring(hz, loud, fades);
  if (inst.unison && ratio < 6) ring(hz * 2 ** (inst.unison / 1200), loud, fades * 0.9);
}
```

Figure 25 Part of modalBuffer(): each mode is a two pole resonator, with stretched overtones and a detuned second string

4.8.4 Recordings

Recorded instruments have a few notes per instrument (rarely every note), so each note is played on the nearest recording, retuned. The Web Audio API retunes a recording by changing its playback rate, which for a change of Δ semitones is:

$$\text{rate} = 2^{\Delta / 12} \tag{11}$$

Changing the playback rate also changes the recording's length and its formants, the resonances that give a voice or instrument its character. Stretched too far, a voice sounds like a chipmunk. A note more than a fifth away from any recording is therefore played synthesised instead. Held notes (strings, organs, choirs) loop the steady middle of their recording, and each recording was measured and tuned where it loops rather than where it starts, after it was found that a soprano recording was a third of a semitone sharp in its held part. Recordings only load when a song uses them, and each layer picks a different recording of its instrument from the one the song is already playing, so two guitars in one song do not sound identical. The reversed piano is a fun detail: each note is played backwards, started early so that its strike lands exactly as the note ends.

4.8.5 Sung lines: a model of a singer's pitch

Voices were the hardest instruments to make convincing. A sampler restarts its recording on every note, which makes a sung melody sound like a series of separate syllables. Real singers move from one note to the next in one continuous line. The project's `singLine` keeps one voice going through a whole phrase, and moves its pitch the way a singer's pitch moves. This is based on Saitou et al.'s (2005) model of the pitch of the singing voice, which found that a singer's pitch moves like a damped spring: it dips slightly the other way just before a move (preparation), rises towards the new note, overshoots it a little and settles. The overshoot was the part that listeners missed most when it was taken out.

A damped spring released towards a new value follows the step response of a second order system. With damping ratio ζ and natural frequency ω, and $\omega_d = \omega\sqrt{(1-\zeta^2)}$, the share of the move covered at time t is:

$$y(t) = 1 - e^{-\zeta\omega t}\left(\cos\omega_d t + \frac{\zeta}{\sqrt{(1-\zeta^2)}}\sin\omega_d t\right) \tag{12}$$

The project uses $\zeta = 0.55$. The overshoot of a second order system is $e^{-\zeta\pi / \sqrt{(1-\zeta^2)}}$, which for $\zeta = 0.55$ gives $e^{-2.069} \approx 0.126$, or 12.6% of the move. This number appears directly in the code (`0.126`), where it is used to scale the overshoot down for large leaps so that it is never more than 25 cents past the note. The natural frequency is set to $\omega = 2.58 / L$, where L is how long the move should take. This value was chosen so that the spring first reaches the target at exactly $t = L$: substituting $\zeta\omega L = 1.419$ and $\omega_d L = 2.154$ into equation 12 gives $\cos(2.154) + 0.659\sin(2.154) \approx -0.551 + 0.550 \approx 0$, so $y(L) \approx 1$. The move time itself grows with the size of the leap, $L = 0.06 + 0.009 \times |\text{semitones}|$ seconds (capped at 0.2 s and at 40% of either note), which is about 75 ms for a step and 170 ms for an octave. Figure 26 shows the paths produced by the real function.

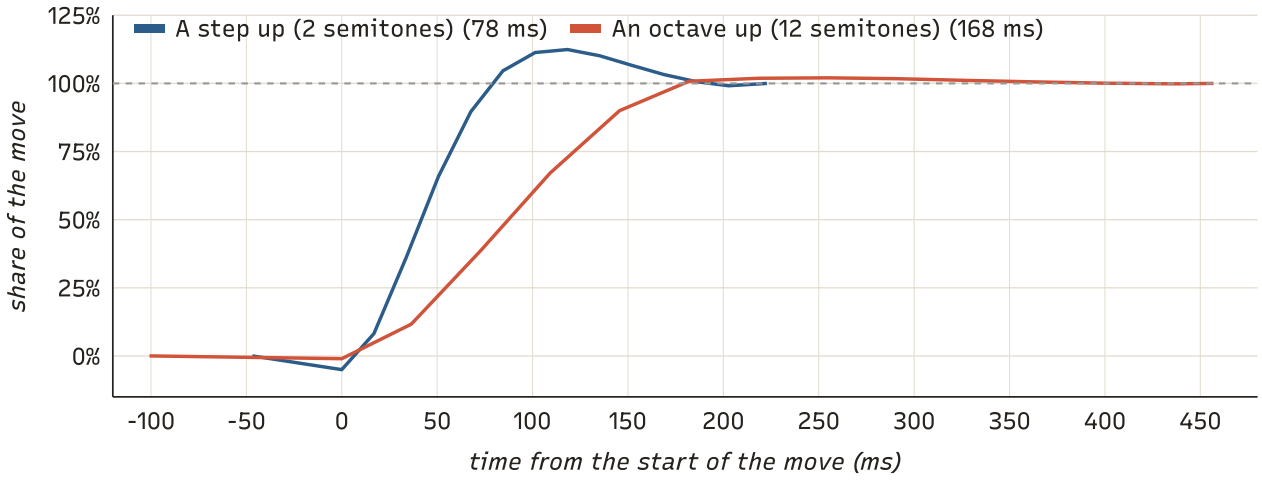


Figure 26 The pitch paths produced by singerMove() for a step and an octave: a small preparation dip, a damped spring onto the note, and an overshoot that settles

```
const DAMPING = 0.55;
function singerMove(from, to, start, length) {
  const leap = to - from;
  const begin = from - Math.sign(leap) * Math.min(12, Math.abs(leap) * 0.05);
  const omega = 2.58 / length;
  const damped = omega * Math.sqrt(1 - DAMPING * DAMPING);
  // a spring over shoots by an eighth of the move, too much for a big leap: past the note, the
swing
  // is scaled down to 25 cents at most (scaled, not cut, which held a flat wrong note)
  const swing = Math.min(1, 25 / (0.126 * Math.abs(to - begin) || 1));
  const points = [[start - length * 0.6, from], [start, begin]];
  for (let k = 1; k <= 12; k++) {
    const t = (k / 12) * length * 2.6;
    const y = 1 - Math.exp(-DAMPING * omega * t) * (Math.cos(damped * t) + (DAMPING / Math.sqrt(1
- DAMPING * DAMPING)) * Math.sin(damped * t));
    points.push([start + t, t < length ? begin + (to - begin) * y : to + (to - begin) * (y - 1) *
swing]);
  }
  points.push([start + length * 2.6 + 0.02, to]);
  return points;
}
```

Figure 27 singerMove(): a damped spring's step response used as the path of a singer's pitch, in cents

To avoid the chipmunk effect, no recording is ever bent more than two semitones: a bigger move starts the nearest recording past its attack and crossfades into it, with both following the same pitch path. For the low voice, the singer Hadzi-Fia was recorded moving from each note into the others, so where a recorded move of the same leap exists, the line plays the singer's real transition, breath and all, before handing over to the held note. Of her 396 recorded transitions, 287 were clean enough to use. In a choir, every voice wanders a few cents of its own and each doubler comes in a touch late, from its own point in the recording, so the choir sounds like people rather than one recording copied.

4.8.6 Guitar rigs and the mixing desk

Distorted guitars go through an amp rig modelled on a real signal chain: a cut before the clipping, a mid push, a compressor for sustain, a waveshaper for the clipping, tone controls, a steep roll off for the speaker, and then double tracking, chorus or reverse reverb. The shoegaze moods add pedals: a tremolo arm that bends the whole chord in and out of tune together, a wah, an octave fuzz, a phaser, a tape flange, and a shimmer (the sound shifted up an octave by two swept delay lines into a wash that feeds back into itself).

Every layer then goes through its own channel on a **mixing desk**, shown in figure 28. The tune is given presence (a boost at 2.5 kHz) and air (a shelf above 8 kHz), and the backing parts give theirs up, by amounts set per role in `PLACES`. This puts the tune in front by tone rather than volume, which is how mixing engineers usually do it. Every part except the bass, drums and drones has its lows cut so that low ends do not pile up into boom. Every instrument shares

one short room (a convolution with hard early reflections in the first 40 ms), and the reverb and echo returns lose their lows on the way back.

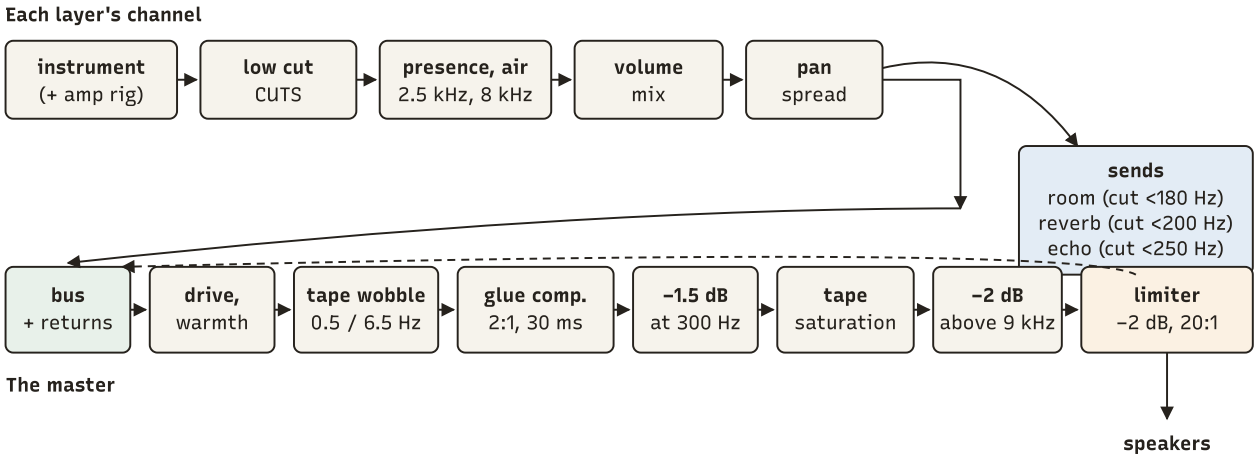


Figure 28 The mixing desk: each layer's channel, its sends to the shared room, reverb and echo, and the master chain

The master chain finishes the sound. "Wobble" imitates worn tape by modulating a short delay with two slow oscillators, wow at 0.5 Hz and flutter at 6.5 Hz. A delay changing over time shifts pitch: if the delay is $\tau(t) = \tau_0 + A\sin(2\pi f_w t)$, the pitch moves by up to a factor of $1 \pm 2\pi f_w A$. At full wobble, the wow's depth of 4 ms gives $2\pi \times 0.5 \times 0.004 \approx 1.3\%$, about 22 cents, which is the gentle seasickness of an old cassette. A glue compressor (2:1 with a slow 30 ms attack) rides the whole band rather than squashing every hit, a small dip at 300 Hz clears mud, and tape saturation rounds off the peaks. The tape curve is a hyperbolic tangent pushed slightly off centre, which adds even harmonics (warmth), and scaled so that its slope at zero is exactly 1, so quiet sounds pass through unchanged and only the peaks are rounded:

$$c(x) = \frac{\tanh(k(x + b)) - \tanh(kb)}{k\,(1 - \tanh^2(kb))}, \quad k = 1.1, \;\; b = 0.1 \tag{13}$$

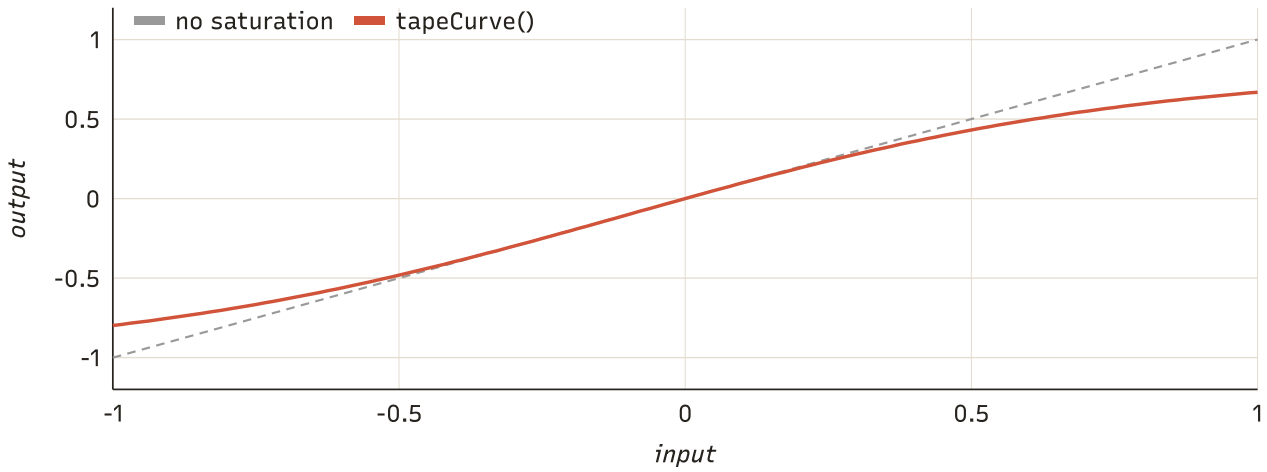


Figure 29 The tape saturation curve, tapeCurve(): unchanged near zero, rounding off the peaks, and slightly lopsided for warmth

The curve is stored with an odd number of points (2049), for a reason that took a long time to find and is described in section 5.3. A limiter at the very end makes sure a loud chorus can never clip.

4.9 Building the sample library

The recordings come from free sample libraries, which disagree on almost everything: file formats, note names, octave numbering, loudness and whether a note named A4 is really an A4. The tool `tools/build-samples.js` builds the library automatically. It downloads only the files it needs (cloning repositories with git where it has to), checks each note's real pitch, trims the silence, levels the notes against each other, and writes small mono MP3s with ffmpeg, plus an `index.json` per folder saying which note each file is and where it came from. The result is 204 folders, about 76 MB in total, which the page fetches only when a song uses them.

The pitch of each note is measured with the YIN algorithm (de Cheveigné and Kawahara, 2002). YIN asks how similar the wave is to a copy of itself shifted by a lag τ, using the squared difference:

$$d(\tau) = \Sigma_j \left(x_j - x_{j+\tau} \right)^2, \quad d'(\tau) = \frac{d(\tau) \cdot \tau}{\Sigma_{j=1}^{\tau} d(j)} \tag{14}$$

If the wave repeats every τ samples, $d(\tau)$ dips towards zero. Dividing by the running average (the "cumulative mean normalised difference", d') stops the algorithm from choosing a tiny lag just because the wave changes little over a few samples. The first lag where d' falls below 0.15 is taken, followed down to the bottom of its dip, and the pitch is the sample rate divided by that lag. The search covers 25 Hz to 4.5 kHz. Since the start of a note is noisy and an overtone can fool the algorithm by an octave, the pitch is measured at five

points after the attack (0.08, 0.15, 0.25, 0.35 and 0.5 seconds) and the median is kept.

```
// the pitch in hz of size samples from at, or null if there's no clear one
function pitchAt(data, at, size) {
  const minLag = Math.floor(RATE / 4500);
  const maxLag = Math.floor(RATE / 25);
  if (at + size + maxLag >= data.length) return null;
  const diff = new Float32Array(maxLag + 1);
  for (let lag = 1; lag <= maxLag; lag++) {
    let sum = 0;
    for (let i = 0; i < size; i++) {
      const d = data[at + i] - data[at + i + lag];
      sum += d * d;
    }
    diff[lag] = sum;
  }
  let running = 0;
  const norm = new Float32Array(maxLag + 1).fill(1);
  for (let lag = 1; lag <= maxLag; lag++) {
    running += diff[lag];
    norm[lag] = running ? (diff[lag] * lag) / running : 1;
  }
  let best = -1;
  for (let lag = minLag; lag < maxLag; lag++) {
    if (norm[lag] < 0.15) {
      while (lag + 1 < maxLag && norm[lag + 1] < norm[lag]) lag++;
      best = lag;
      break;
    }
  ...
```

Figure 30 pitchAt() in tools/build-samples.js: the YIN pitch detector used to check every recorded note

YIN is not perfect. A nyunga nyunga mbira and one set of tubular bells were left out of the library entirely because their buzzing and their bells' inharmonic overtones fooled every pitch check, so their tuning could not be trusted. It is also a good example of why the tool checks the pitch rather than trusting the file names: libraries disagree on octave numbering, so every new folder is first run with a `--probe` option that reports what it measured before anything is written.

4.10 The learn tab: a guitarist's hand and the Viterbi algorithm

The Learn tab shows any part of the song the way a player would read it: falling notes on piano keys (the right hand green and the left blue, like a piano cover video), a guitar or bass neck with numbered fingers over a scrolling tab, or sheet music. Figure 31 shows a shoegaze guitar part.

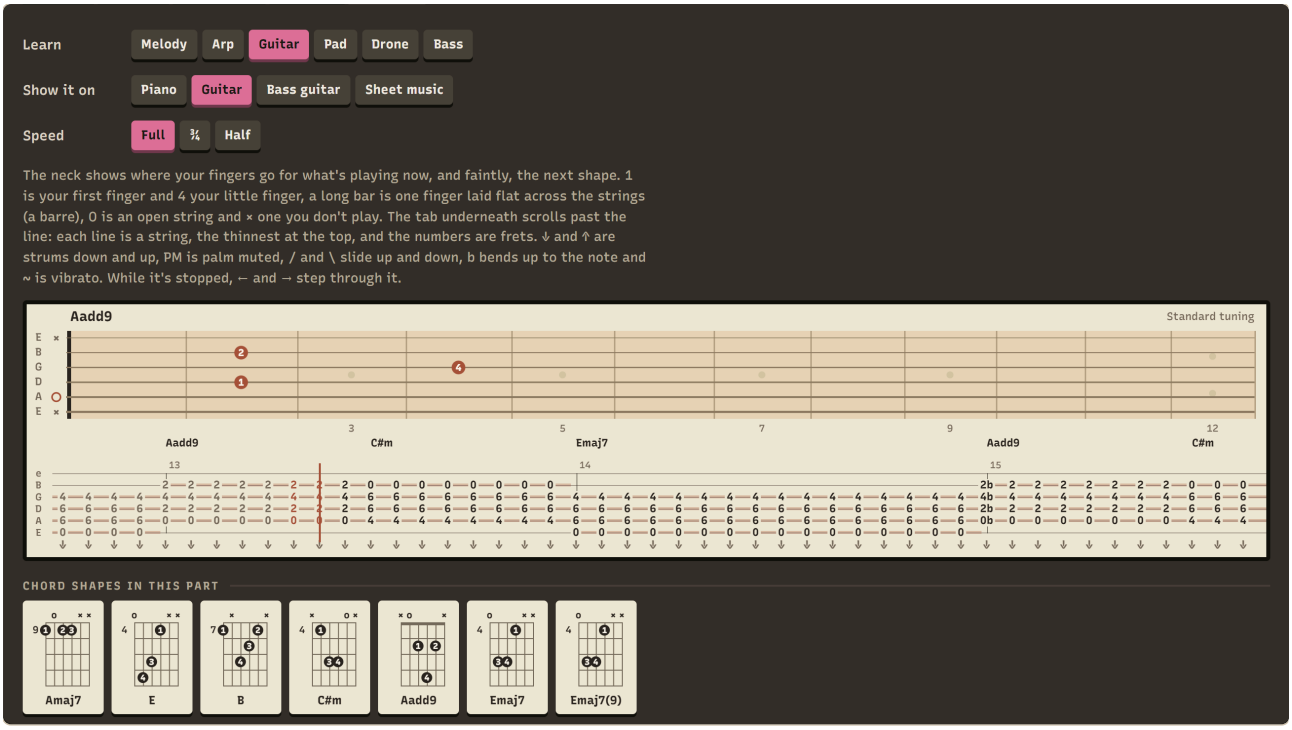


Figure 31 The Learn tab showing a guitar part: the fingers for the current chord on the neck, the scrolling tab, and the chord shapes used in the part

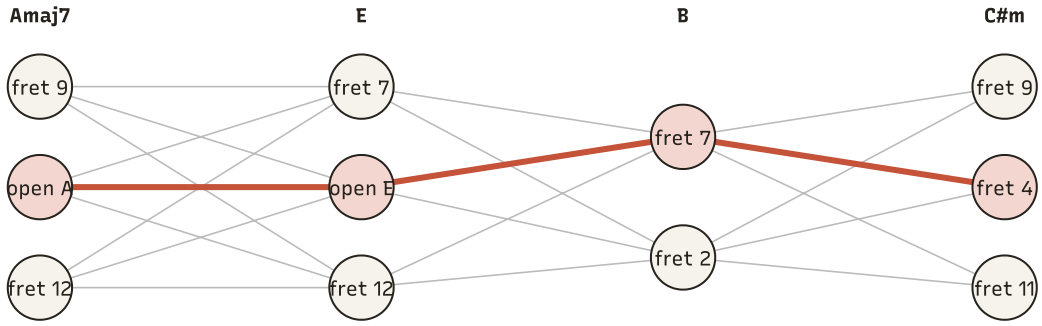
Showing a guitar part is much harder than showing a piano part, since the same note can be played in several places on a guitar's neck and a chord can be held in many ways. The first step is a model of a hand. `holds()` finds every way to place a set of pitches on the strings, one note per string, within the reach of a hand (four frets, or three below the fifth fret where the frets are wider). `fingering()` then decides whether a hand can actually hold each way, and how awkward it is. It has four fingers numbered up the frets, one fret each. A finger can lie flat across several strings (a barre), but only over strings that are held higher, since it would stop any open or lower string underneath: the first finger can barre any number of strings, the third can barre three (the "A shape"), and the others two. The cost adds up stretches, fingers bunched on one fret and barres.

The second step is choosing one way for every chord or note in the part. Choosing the cheapest way for each chord on its own sends the hand jumping up and down the neck. Instead, the whole part is solved at once with the Viterbi algorithm (Viterbi, 1967), the same dynamic programming method used to decode signals and to find the most likely path through a hidden Markov model (Rabiner, 1989). Sayegh (1989) first applied this "optimum path" idea to fingering for string instruments. If $C(i, k)$ is the cheapest total cost of playing events 1 to i with event i held its kth way:

$$C(i, k) = \text{own}(i, k) + \min_j \left[C(i - 1, j) + \text{move}(j \to k) \right] \tag{15}$$

where own is how awkward a way is to hold and move is how far the hand travels between two ways. Remembering which j gave each minimum and

walking back from the cheapest final state gives the best path. For n events with up to K ways each, this costs $O(nK^2)$, rather than the $O(K^n)$ of trying every combination. Figure 32 illustrates the idea.



each column is a chord and each circle a way to hold it; the red line is the cheapest path through the whole part

Figure 32 An illustration of the Viterbi search in the learn tab: every way to hold each chord, with the cheapest path through the part chosen at once

```
// viterbi: best[i][k] the cheapest way to reach event i held its kth way
let best = [];
const back = [];
options.forEach(({ ways, costs }, i) => {
  const row = ways.map((way, k) => {
    const own = costs[k];
    if (!i || !best.length) return { cost: own, from: -1 };
    let pick = { cost: Infinity, from: -1 };
    best.forEach((previous, j) => {
      const cost = previous.cost + moveCost(options[i - 1].ways[j], way) + own;
      if (cost < pick.cost) pick = { cost, from: j };
    });
    return pick;
  });
  back.push(row);
  best = row;
});
const chosen = new Array(options.length).fill(-1);
let k = best.reduce((at, one, j) => (one.cost < best[at].cost ? j : at), 0);
for (let i = options.length - 1; i >= 0 && k >= 0; i--) {
  if (!options[i].ways.length) break;
  chosen[i] = k;
  k = back[i][k].from;
}
```

Figure 33 The Viterbi search in frettings() (learn.js), choosing strings and frets for a whole part at once

Building this revealed that the composer had been writing chords no hand could hold, as described in section 3.5.6. The same hand model is now shared by the composer and the learn tab. When the composer gives a guitar a chord, `forGuitar()` checks it; a chord that can be held stays as written, and one that cannot is moved to the nearest chord that can, keeping the same bass note, the same top note and the close "rubs" between neighbouring notes where possible, and leaving out the chord's fifth first if a note has to go, as a guitarist

would. Measured across every mood, flavour and style, the number of notes the learn tab had to leave off fell from 17,784 to 5 out of about 106,000 chords (the 5 being melody runs faster than the tab's grid).

The other views use simpler rules. The piano view gives the left hand the bass line by default, like a piano cover. The sheet music view spells each note using the key signature of the major key that shares the most notes with the song's scale (a mode shares all seven with its own, so a B lydian song has F sharp major's notes and is written with six sharps, as in figure 34) and cuts notes at bar lines into tied values. A practice speed setting plays a slower copy of the song without changing the song itself.

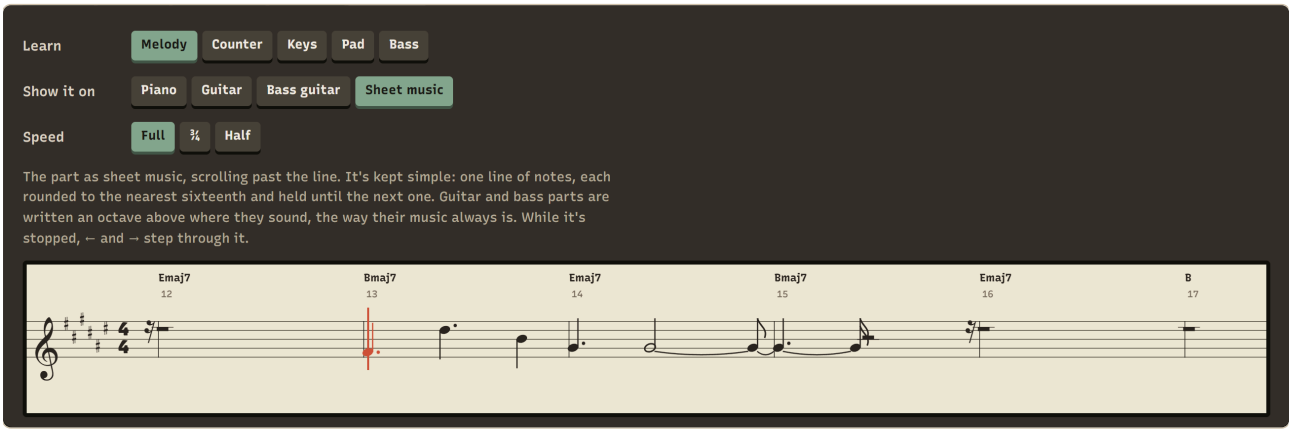


Figure 34 The Learn tab's sheet music view, with the key signature chosen from the song's scale and notes tied across bar lines

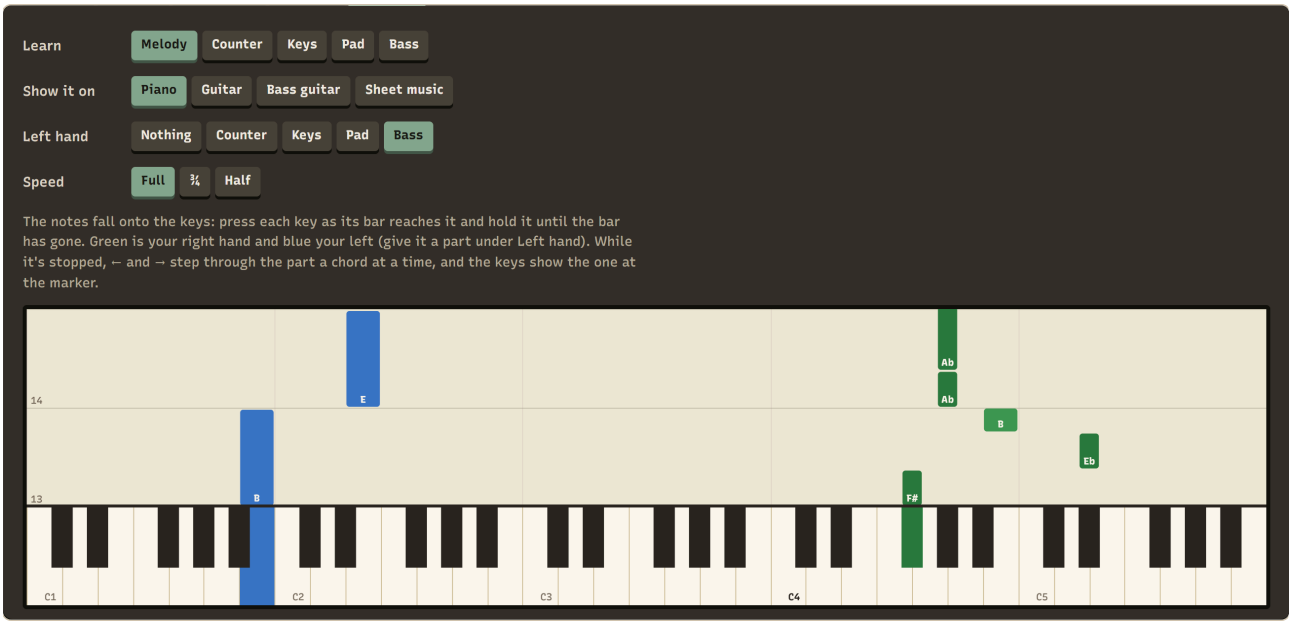


Figure 35 The Learn tab's piano view, with notes falling onto the keys: green for the right hand and blue for the left

4.11 Saving, sharing and keeping old songs

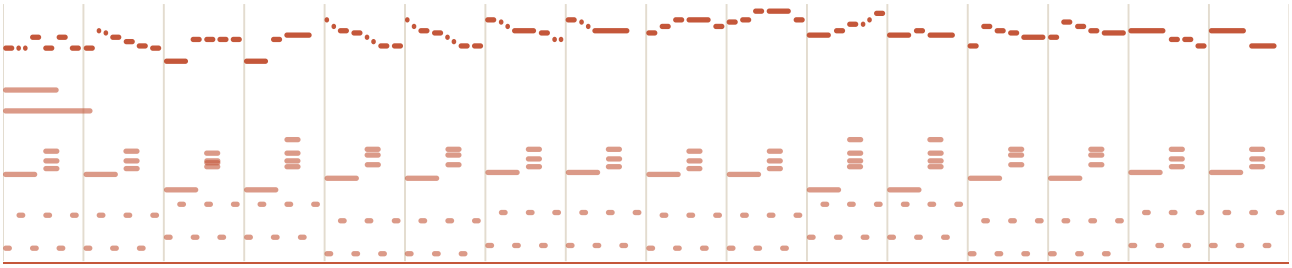
4.11.1 Saved songs never change

As explained in section 3.5.5, storing a song as settings means a change to the composer changes the notes of every saved song. The solution has three parts. First, every song is stamped with the composer that made it (`song.composer`). Second, before any change that would make saved songs sound different, a tool (`tools/keep-composer.js`) copies today's `moods.js` and `music.js` into `kept/composer-n.js`, wrapped in a closure so their names do not clash, and registers it. When a song made by an older composer is played, `composeSong` hands it to that composer. There are five composers so far (1, 1.5, 2, 3 and 4 kept, and 5 live). Third, a check (section 5.2) keeps about two hundred songs from each composer and fails if any of them would play a single note differently.

Saved songs are kept in IndexedDB, with a copy in localStorage. Each save keeps the settings and the notes they played, deflated and fingerprinted. Saving over a song moves the old version into its history (forty versions back), deleting moves it into a bin, and a backup is every song, history and bin as one file. Restoring a backup only ever adds songs, never removes them, so a mistake can never lose anything.

4.11.2 Short links

A song can be shared as a link. The first version of links put the whole song into the address as base64, which made them far too long to share. Links are now written as `#seed/mood/style/modifiers/flavour/changes`: what the seed makes, plus only what has been changed since (`songChanges()`), compressed with the browser's own deflate (`CompressionStream('deflate-raw')`) and written in URL safe base64. An unchanged song's link is just its words, for example `#mossy-lantern/nostalgic/tune`, and a song made by hand is the changes from a blank track.



Bars 57 to 72 of "cobwebbed-sandman" in the Tense mood, as the composer writes them

5 Testing and Evaluation

5.1 Overview of the testing approach

Testing a music generator is unusual, since the most important question ("does it sound good?") can only really be answered by listening. However, a large number of the problems found during development turned out to be things that could be measured: a melody note clashing with a chord, two layers on the same instrument, a guitar chord no hand can hold, a recording out of tune. The project therefore relies on a combination of automated checks, measurement tools that run in the browser, testing the page with a real mouse and keyboard, and listening. A rule followed throughout development was that every fix for something that sounded wrong should, where possible, become an automated check, so that it can never quietly come back.

5.2 Automated checks

Since the composer has no page and no audio, it can be loaded straight into Node.js and run thousands of times. Figure 36 lists the checks.

Check	Time	What it checks
`music-check.js`	about 5 minutes	Every mood, style and flavour, in thirty five blocks: grooves, a band that fits, counterpoint, ornaments, orchestration, weaves, builds, key changes, short links, that seeds give the same song, that recordings exist and are chosen, and more.
`saved-songs-check.js`	about 1 minute	About two hundred songs from each composer still play exactly the notes they were saved with.
`learn-check.js`	about 30 seconds	Common chord shapes are fingered as players finger them, shapes no hand holds are refused, every guitar chord in every mood and flavour can be held, bars add up in the sheet music.
`library-check.js`	a few seconds	Notes pack and unpack, saving over keeps history, names never clash, and backups only ever add.
`tuning.html`	about 1 minute	Every held recording is within 10 cents of its note where it loops.

Check	Time	What it checks
`levels.html`	about 1 minute	How loud every instrument and texture is, weighted the way ears hear, so new sounds can be levelled against the others.

Figure 36 The project's automated checks and measurement pages

The main check, `music-check.js`, generates songs for every combination of mood, flavour and style and asserts statistical properties of the results rather than exact notes. For example, it asserts that more than 85% of melodies start their bars on a chord note, that no two layers share an instrument, that no line runs faster than six notes a second, and that a counter melody set to counterpoint never moves in parallel fifths. A block that fails does not stop the others, so a single run lists every failure. Words after the command run only the blocks whose names contain them, which takes seconds and is how I checked the part I had just changed. Figure 37 shows example output.

```
> node js/squimble-music-maker/tests/music-check.js counterpoint seeds "short links"
all passed in 1 s (3 pieces; slowest: counterpoint 1 s, short links 0 s, seeds 0 s)

> node js/squimble-music-maker/tests/library-check.js
library-check passed
```

Figure 37 The output of a few of the music check's blocks and the library check, run while writing this report

An interesting practical problem was that the full music check originally used every core of the computer flat out for several minutes, which made the computer lag badly while anything else was running. It was changed to run in pieces side by side on half of the cores at a low priority, with the saved songs check running beside it, so the computer stays usable while it runs.

Determinism is also checked directly. Generating and composing the song in figure 1 twice in a row gives an identical list of 565 notes, and the saved songs check proves the same is true across composers and across time. In Node.js, generating and composing a song takes about 26 milliseconds on average, so writing the music itself is never what makes a page slow.

Finally, the page was tested in a real browser with Playwright, using real mouse clicks, key presses and taps at a phone's width as well as a computer's, checking that each feature worked and that the page threw no errors. A small tool (`tests/headless.js`) runs the measurement pages in headless Chrome over the DevTools protocol, so levels can be measured without opening a window.

5.3 Performance

All of the sound is made live in the browser's audio thread, which must produce each block of 128 samples before the speakers need it. If any block takes too long, the sound stutters. Large songs, such as a shoegaze song with ten layers of distorted guitars, began to stutter on a computer, so the audio

thread was measured using Chrome's tracing tools, which record how long each block of audio takes to render. Measuring offline (rendering a song as fast as possible) was tried first but turned out to be too noisy to show small effects.

The measurements revealed two surprising causes. The first was that Chrome's "balanced" setting for audio latency gives a 10 ms buffer on Windows, the same as its lowest latency setting, so any block that ran slightly long became an audible stutter. Asking for a 40 ms buffer gave the audio thread room to absorb the occasional slow block. The playhead and the keyboard recording were set back by the browser's reported output latency so that they still match what is heard.

The second cause was much harder to find. A Web Audio `WaveShaperNode` maps an input x between −1 and 1 to a position in its curve, $(N-1)(x+1)/2$ for a curve of N points, interpolating between neighbouring points. With an even number of points, silence ($x = 0$) lands halfway between the two middle points. For a symmetrical curve this does not matter, but the tube, octave fuzz and tape curves are deliberately lopsided (for warmth), so silence came out as a tiny steady signal. As a result, every idle guitar rig, its channel and the reverb behind it never received true silence and never got to rest. Making every curve an odd length (such as 2049 points) puts $x = 0$ exactly on the middle point, where the curve is zero, so silence stays silent. Figure 38 summarises the result.

Measurement (a ten layer shoegaze song)	Before	After
Audio thread load in the busy part	about 75%	about 58 to 60%
Blocks that ran over time	16 to 95 in 20 seconds	9 in 172 seconds (of 40 ms callbacks)
Silent cost of a chorus guitar rig	0.14	0.045
Audio buffer on Windows	10 ms	40 ms

Figure 38 The effect of the performance changes on a large song, measured with Chrome's tracing of the real time audio thread

Earlier rounds of performance work had already shortened the reverb's tail (stopping it 45 dB down, since its length was the single biggest cost), started the song's clock only once the band was ready, worked out plucked and struck notes ahead of time, and given phones and tablets a bigger buffer, notes sent further ahead and a shorter reverb. On iPhones, a page's Web Audio is silenced by the phone's silent switch unless the browser treats it as media. Newer versions of Safari can be told directly (`navigator.audioSession.type = 'playback'`), and on older ones the page loops a second of silence through an HTML audio element alongside the music, which switches the phone over.

5.4 Feedback and evaluation against the objectives

The project has not yet been through a formal user study like the one carried out for my previous project. Feedback so far has been informal, from friends

trying the page and from my own listening. The most significant piece of feedback was that friends found the page complicated and cluttered, especially on a phone. This led to the layout described in section 3.4: a shorter introduction, a welcome card, the style given its own row, a tip line docked to the bottom of the window, controls that fold away, and swipeable rows on phones. It should also be acknowledged that a lot of the sound (the levels of recordings, the mixing desk, doubling, the voices and the key changes) was set from measurements, printed notes and the automated checks rather than by ear, and some features have not been listened to carefully yet. Figure 39 evaluates the project against its objectives.

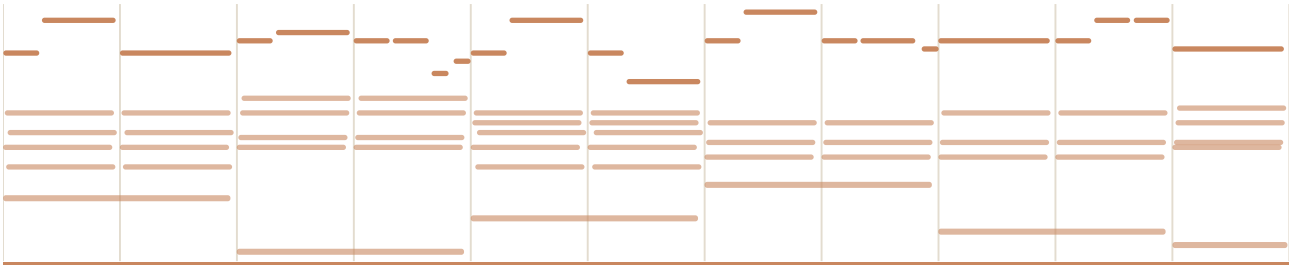
Objective	Met?	Evidence
Generate a complete song from a seed and mood, the same every time	Yes	generateSong and composeSong; the seeds check and identical notes when run twice
A composer with no page or audio, runnable in Node.js and the game	Yes	music.js, moods.js and instruments.js load in Node.js; every check runs there
A band that sounds like real players	Mostly	Physical models, recordings, a mixing desk and ornaments; some sounds still set by measurement rather than by ear
Change, save and share songs without a server	Yes	Every setting and note editable; IndexedDB library; short links; song files
Saved songs never change	Yes	Kept composers and the saved songs check across five composers
Usable on phones, and teaches its parts	Yes	The phone layout and the Learn tab; confirming that friends find the new layout easier is still to do

Figure 39 An evaluation of the project against the objectives set in section 1.3

5.5 Limitations

Despite this, there are several known limitations, which are recorded in the project's documentation rather than hidden.

- Listening only compares parts against the melody, so two backing parts can still rub against each other, and textures do not get quieter under the tune.
- Only four time signatures are supported (4/4, 3/4, 6/8 and 5/4), since a bar must be a whole number of beats.
- Some recordings stand in for instruments that no free library has recorded, and the voices have only a few notes each, so a sung note can be up to two semitones from its nearest recording.
- A shared link stores the seed and its changes, so it opens with whatever the composer on the page picks for that seed at the time. A saved song or a song file, by contrast, keeps its notes.
- The learn tab's sheet music is a single line of notes rounded to sixteenths, and piano parts are not fingered.
- About one solo in seven ends on a chord note other than the root.



Bars 29 to 39 of "swaying-pie" in the Cozy mood, as the composer writes them

6 Conclusion

This project set out to create a music generator that could write the soundtrack for a video game using the rules of music theory rather than machine learning, while also being a tool that people can use to make, change, save and learn music. Through the combination of seeded random numbers, music theory written as data and cost functions, and a band built from physical models and recordings, the project has largely achieved this. It can write songs in sixty two moods and over two hundred flavours, in seven styles named after what a game needs, and the same seed always produces the same song.

From a technical perspective, the most valuable decision was separating the composer from the band and keeping the composer free of the page and of audio. This made it possible to test the composer thoroughly in Node.js, to keep older composers alive for saved songs, and to load it into the game without changes. The decision to store songs as settings rather than notes made links short and remixing easy, but it created the problem of saved songs changing, which took a careful system of kept composers and checks to solve. The most interesting parts mathematically were the ones borrowed from other fields: Karplus-Strong and modal synthesis from physical modelling, a second order system from control theory for a singer's pitch, YIN from speech processing for tuning the sample library, and the Viterbi algorithm from coding theory for fingering a guitar part.

A recurring lesson was that parts which sound fine alone can sound wrong together. A large share of the project became rules about what goes with what, such as one groove for the band, a cap on how harsh instruments can be, and a band that listens before it plays. Writing every fix down alongside what it sounded like before, and turning most of them into checks, kept these rules from being lost.

Regardless of its limitations, the project has demonstrated that a rule based generator can produce varied, structured and editable music that runs entirely in a web browser, on a phone, with no server. Future work will focus on moving the music maker into Squimble Quest itself, where moods will be chosen by the area of the world and fights will switch to the fight styles. Other planned improvements include looping a single section while editing it,

selecting and moving several notes at once, letting listening compare backing parts against each other, and recording more of a singer's transitions between notes. Finally, in the same way as my previous project, a mixed methods listening study, comparing generated songs with songs written by people using both ratings and open questions, would give a much better answer to the most important question of all: whether the music is good.

References

Agostinelli, A., Denk, T.I., Borsos, Z., Engel, J., Verzetti, M., Caillon, A., Huang, Q., Jansen, A., Roberts, A., Tagliasacchi, M., Sharifi, M., Zeghidour, N. and Frank, C. (2023) 'MusicLM: Generating music from text', *arXiv preprint* arXiv:2301.11325. Available at: https://arxiv.org/abs/2301.11325

Chowning, J.M. (1973) 'The synthesis of complex audio spectra by means of frequency modulation', *Journal of the Audio Engineering Society*, 21(7), pp. 526–534.

Collins, K. (2008) *Game Sound: An Introduction to the History, Theory, and Practice of Video Game Music and Sound Design*. Cambridge, MA: MIT Press.

de Cheveigné, A. and Kawahara, H. (2002) 'YIN, a fundamental frequency estimator for speech and music', *The Journal of the Acoustical Society of America*, 111(4), pp. 1917–1930. Available at: https://doi.org/10.1121/1.1458024

Dhariwal, P., Jun, H., Payne, C., Kim, J.W., Radford, A. and Sutskever, I. (2020) 'Jukebox: A generative model for music', *arXiv preprint* arXiv:2005.00341. Available at: https://arxiv.org/abs/2005.00341

Eno, B. (1996) 'Generative music', *In Motion Magazine*, 7 July. Available at: https://inmotionmagazine.com/eno1.html

Fletcher, H. (1964) 'Normal vibration frequencies of a stiff piano string', *The Journal of the Acoustical Society of America*, 36(1), pp. 203–209.

Fowler, G., Noll, L.C., Vo, K.-P. and Eastlake, D. (2019) *The FNV Non-Cryptographic Hash Algorithm*. IETF Internet-Draft. Available at: https://datatracker.ietf.org/doc/draft-eastlake-fnv/

Hiller, L.A. and Isaacson, L.M. (1959) *Experimental Music: Composition with an Electronic Computer*. New York: McGraw-Hill.

Huron, D. (2006) *Sweet Anticipation: Music and the Psychology of Expectation*. Cambridge, MA: MIT Press.

Jaffe, D.A. and Smith, J.O. (1983) 'Extensions of the Karplus-Strong plucked-string algorithm', *Computer Music Journal*, 7(2), pp. 56–69. Available at: https://doi.org/10.2307/3680063

Karplus, K. and Strong, A. (1983) 'Digital synthesis of plucked-string and drum timbres', *Computer Music Journal*, 7(2), pp. 43–55. Available at: https://doi.org/10.2307/3680062

Lerdahl, F. and Jackendoff, R. (1983) *A Generative Theory of Tonal Music*. Cambridge, MA: MIT Press.

Mann, A. (1965) *The Study of Counterpoint, from Johann Joseph Fux's Gradus ad Parnassum*. Revised edn. New York: W.W. Norton.

Nielsen, J. (1995) *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/

Nielsen, J. (2006) *Progressive Disclosure*. Nielsen Norman Group. Available at: https://www.nngroup.com/articles/progressive-disclosure/

Nierhaus, G. (2009) *Algorithmic Composition: Paradigms of Automated Music Generation*. Vienna: Springer.

Piston, W. and DeVoto, M. (1987) *Harmony*. 5th edn. New York: W.W. Norton.

Plut, C. and Pasquier, P. (2020) 'Generative music in video games: State of the art, challenges, and prospects', *Entertainment Computing*, 33, 100337.

Rabiner, L.R. (1989) 'A tutorial on hidden Markov models and selected applications in speech recognition', *Proceedings of the IEEE*, 77(2), pp. 257–286. Available at: https://doi.org/10.1109/5.18626

Saitou, T., Unoki, M. and Akagi, M. (2005) 'Development of an F0 control model based on F0 dynamic characteristics for singing-voice synthesis', *Speech Communication*, 46(3–4), pp. 405–417.

Sayegh, S.I. (1989) 'Fingering for string instruments with the optimum path paradigm', *Computer Music Journal*, 13(3), pp. 76–84.

Smith, J.O. (2010) *Physical Audio Signal Processing*. W3K Publishing. Available at: https://ccrma.stanford.edu/~jos/pasp/

Tymoczko, D. (2011) *A Geometry of Music: Harmony and Counterpoint in the Extended Common Practice*. New York: Oxford University Press.

Viterbi, A.J. (1967) 'Error bounds for convolutional codes and an asymptotically optimum decoding algorithm', *IEEE Transactions on Information Theory*, 13(2), pp. 260–269. Available at: https://doi.org/10.1109/TIT.1967.1054010

W3C (2021) *Web Audio API*. W3C Recommendation, 17 June 2021. Available at: https://www.w3.org/TR/webaudio/

Appendices

A Moods and flavours

Every mood in the music maker, by the group it is shown under on the page, with its flavours as the page names them (the first flavour of each mood is the plain mood). This table was generated from `moods.js` when this report was made.

Group	Mood	Flavours (page names)
Gentle	Calm	Meadow, Lullaby, Zen garden
	Heavenly	Clouds, Cathedral, Ascension
	Wistful	Memories, Autumn, Countryside
	Nostalgic	First days, Embers, Tinkering, Slow build, Lullaby, Rolling keys, Circuits
Bright	Happy	Spring, Island life, Festival
	Playful	Romp, Tiptoe, Big top
	Spirited	Adventure, Keepsake, Old caverns
	Upbeat	Full speed, Dance floor, City pop
	Cozy	Fireside, Tea shop, Rainy day
	Heroic	Quest, Fanfare, Space opera
Magic	Magic	Wizard school, Fairy glade, Forbidden, Blossom, Starlit tides
	Gaslamp	City waltz, Back alleys, Music hall, Clockwork, Moonlit manor
	Dragon	Burning citadel, Lair, Charge
	Sands	Pyramids, Royal march, Golden plains
	Jade	Jade gardens, Blight, War drums, Mists
	Greenwood	Greenwood, Fae wood, Longhouse
Easy	Jazz	Bebop, Ballad, Big band, Noir
	Lounge	Bossa nova, Lift music, Exotica
	Soul	Late night, Sixties soul, Gospel
	Showtime	Spotlight, Disco, Game show
	Shop	Laid back, General store, Black market
	Blues	Shuffle, Delta, Electric
	Country	Country road, Bluegrass, Campfire
Dark	Sad	Sorrow, Requiem, Alone
	Tragic	Lament, Poisoned sea, Music box, Last stand
	Bleak	Fuzz wall, Post-rock, Hymn
	Mysterious	Secrets, Lost temple, Moonlit
	Tense	Danger, Stealth, Countdown
	Eerie	Haunting, Dead carnival, Static, Fog, Rust, Regret
	Desolate	Ashes, Bunker, Ruined city

Group	Mood	Flavours (page names)
Beats	Neon	Night club, Rain city, Industrial
	Synthwave	Outrun, Darksynth, Dreamwave
	Breakbeat	Liquid, Jungle, Trip hop
	Pop	Radio, Power ballad, Bubblegum
	Hip hop	Beat tape, Lo-fi study, Jazz hop, Golden age
	Trap	Dark trap, Cloud rap, Drill
World	Desert	Bazaar, Oasis, Palace
	Tropical	Island, Jungle, Hawaii
	Folk	Village, Tavern, Slow air
	Wintry	Snowfall, Snow town, Christmas, Ice cave
	Far east	Temple, Festival, Ninja, Tea house
	Raga	Raga, Dawn, Filmi
	Reggae	Roots, Dub, Ska
	Steppe	Plains, Gallop, Yurt
	Gamelan	Court, Kebyar, Night garden
	Flamenco	Tablao, Rumba, Soleá
Ancient	Aztec	Sun temple, Ritual, Rainforest
	Tribal	Firelight, Savanna, War dance
	Egyptian	The Nile, Tomb, Pharaoh
	Antiquity	Agora, Legion, Olympus
	Medieval	Market, Court dance, Chant
Places	Wasteland	Frontier, High noon, Old radio
	Underwater	The deep, Reef party, Lagoon, Sunken city
	Pirates	Shanty, Swashbuckler, Calm seas, Ghost ship
	Space	Drift, Space station, Alien world
	Iron city	Tyrant's palace, Pest control, Factory floor, Prison
Loud	Metal	Thrash, Symphonic, Stoner
	Grunge	Garage, Unplugged
	Hardcore	Melodic, Pop punk, Twinkle
	Shoegaze	Wall of sound, Blackgaze, Rush, Undertow, Ache, Concrete, Velvet, Sunlit, Melt, Screen glow
	Dream pop	Haze, Bedroom pop
	Ethereal	Soaring, Dancing, Mellow

B Glossary

Cent	A hundredth of a semitone. Most listeners notice a difference of around 5 to 10 cents between two notes played together.
Chord progression	A sequence of chords. Written here as roman numerals, so I is the chord on the first note of the key and V the chord on the fifth.
Contour	The shape of a melody: which way each note moves and by how far.
Counterpoint	Two or more independent melodies played together, following rules that keep them sounding good together.
Formant	A resonance of the voice or an instrument's body, which stays at the same frequency whatever note is played and gives the sound its character.
Groove	The rhythm the drums, bass and chord stabs share in a bar.
Harmonic	A frequency at a whole multiple of a note's frequency. The mix of harmonics decides an instrument's tone.
Mode	In music theory, a kind of scale (dorian, lydian). In synthesis, one of the frequencies a struck object rings at.
Motif	A short musical idea, a rhythm and a shape, that a melody is built from.
Seed	The two words every random choice in a song comes from.
Swing	Playing the off beats late, giving a lilting feel.
Voice leading	How each note of one chord moves to a note of the next.